DOKUZ EYLUL UNIVERSITY

CME 2210 OBJECT ORIENTED PROGRAMMING

Allergy Aware Grocery System

Lecturer: Prof. Dr. Semih Utku

By:
2023510230 - İpek Dalgıç
2023510180 – Kadriye Beren Sümer
2023510138 – Sara Nimet Coğuplugil

1. Introduction

1.1 Project Objectives

2. Problem Definition

3. Requirements

3.1 Functional Requirements

3.2 Non-Functional Requirements

3.3 Use Cases

3.3.1 Use Case Descriptions

3.3.2 Actors

3.3.3 Use Case Summary

3.3.3.1 UC-01: Register and Create Allergy Profile

3.3.3.2 UC-02: Browse Products and Add Product to Cart with Allergy Check

3.3.3.3 UC-03: Checkout and Payment Processing

3.3.3.4 UC-04: Admin Manages Products and Order Statuses

4. Architectural Design

4.1 Architectural Overview

4.2 Presentation Layer

4.3 Application / Service Layer

4.4 Domain Layer

4.5 Infrastructure Layer

4.6 Dependency Direction

4.7 Manual Dependency Injection

4.8 Architectural Benefits

4.9 Implementation Overview and Code Structure

4.10 Key Classes and Component Interaction

5. UML Diagrams

5.1 Use Case Diagram

5.2 Package Diagram

5.3 Class Diagram

5.4 Sequence Diagrams

5.4.1 Sequence Diagram – Register and Create Allergy Profile

5.4.2 Sequence Diagram – Browse Products and Add Product to Cart

5.4.3 Sequence Diagram – Checkout and Payment Processing

5.4.4 Sequence Diagram – Admin Updates Order Status

5.5 Statechart Diagram

5.6 Activity Diagrams

5.6.1 Activity Diagram – Allergy-Aware Add to Cart

5.6.2 Activity Diagram – Checkout

6. Design Patterns Used

6.1 Overview of Applied Design Patterns

6.2 Singleton Pattern – ApplicationContext

6.3 Builder Pattern – CustomerBuilder

6.4 Factory Method Pattern – ProductFactory

6.5 Strategy Pattern – AllergyCheckStrategy

6.6 Facade Pattern – CheckoutFacade

6.7 Adapter Pattern – PaymentGatewayAdapter

6.8 Observer Pattern – OrderObserver

6.9 State Pattern – OrderState

6.10 Summary of Pattern Usage

7. SOLID Evaluation

7.1 Single Responsibility Principle

7.2 Open-Closed Principle

7.3 Liskov Substitution Principle

7.4 Interface Segregation Principle

7.5 Dependency Inversion Principle

7.6 General SOLID Summary

8. Testing Strategy

8.1 Overview

8.2 Customer Registration and Login Testing

8.3 Product Browsing and Allergy Check Testing

8.4 Cart Operation Testing

8.5 Checkout and Payment Testing

8.6 Order Lifecycle and State Transition Testing

8.7 Admin Operation Testing

8.8 Input Validation and Edge Case Testing

8.9 Testing Summary

9. Limitations

9.1 Overview

9.2 Console-Based User Interface

9.3 In-Memory Data Storage

9.4 Basic Authentication

9.5 Simulated Payment Processing

9.6 Basic Allergy Checking Logic

9.7 Limited Product and Inventory Features

9.8 No Multi-User or Concurrency Support

9.9 Limited Error Logging and Monitoring

9.10 Testing Limitations

9.11 Limitations Summary

9.12 Additional Discussion: Complexity, Maintainability, and Architectural Choices

9.12.1 Complexity Discussion

9.12.2 Maintainability and Extensibility Discussion

9.12.3 Manual Dependency Injection

9.12.4 Alternative Architectural Choices

9.13 Reflection and Future Improvements

10. Conclusion

Appendix A

Appendix B

1. Introduction

In modern e-commerce systems, users usually make purchasing decisions by browsing products, adding items to a cart, and completing payment through a checkout process. However, for some users, especially customers with food allergies, a normal product catalog is not enough. A grocery shopping system should not only list products and prices, but should also help the customer make safer decisions by considering allergen information. This project, named **Allergy Aware Grocery System**, is designed around this need. It provides a console-based grocery shopping application where customers can register, define their allergy profiles, browse products, receive allergy warnings, manage their carts, and complete orders through simulated payment methods.

The selected project domain is related to e-commerce order processing, but it is extended with allergy awareness. The system does not only focus on the basic shopping flow. It also checks whether the allergens of a selected product conflict with the customer's allergy profile. If a risky product is selected, the system warns the customer before the product is added to the cart. During checkout, allergy warnings are also considered again so that the customer makes a conscious decision before completing the order. This makes the system more meaningful than a simple shopping cart application, because it includes decision support and user safety as part of the business logic.

The main purpose of this project is not to build a visually advanced user interface or a real database-backed commercial application. Instead, the main purpose is to demonstrate object-oriented analysis and design principles in a medium-scale Java project. For this reason, the system is implemented as a console application, but its internal design follows a layered architecture. The project separates responsibilities into presentation, application/service, domain, and infrastructure layers. This separation makes the code easier to understand, maintain, test, and extend. The console menus are responsible only for interacting with the user, while the service classes coordinate business operations. The domain layer contains the main business concepts such as customer, product, cart, order, payment request, allergy profile, and order status. The infrastructure layer provides in-memory repository implementations, payment gateway adaptation, seed data, and application configuration.

The project also aims to show how design patterns can be applied in a realistic but understandable way. Several patterns are used to solve different design problems. For example, the **Builder Pattern** is used while creating customer objects in a controlled way. The **Factory Method Pattern** is used for product creation. The **Singleton Pattern** is used in the application context to create and share the main dependencies of the system. The **Strategy Pattern** is used for allergy checking behavior, which allows the

allergy checking logic to be changed or extended without rewriting the cart service. The **Facade Pattern** simplifies the checkout process by combining cart validation, order creation, payment processing, stock update, and cart clearing under a single checkout operation. The **Adapter Pattern** connects the application's payment service with an external payment gateway simulation. The **Observer Pattern** is used for order status notifications and logging. The **State Pattern** is used to control valid order status transitions such as created, paid, preparing, shipped, delivered, and cancelled.

Another important goal of the project is to evaluate the system according to SOLID principles. The code is designed so that each class has a clear responsibility. For example, customer-related operations are handled by CustomerService, cart-related operations are handled by CartService, payment operations are handled by PaymentService, and order lifecycle operations are handled by OrderService. Repository interfaces are defined in the domain layer, while their in-memory implementations are placed in the infrastructure layer. This supports dependency inversion because application services depend on abstractions rather than directly depending on concrete repository classes. At the same time, the project still has areas that can be improved, such as adding more advanced validation classes, using dependency injection more flexibly, or replacing the console interface with another presentation layer in the future.

The implemented system supports two main user roles: customer and admin. Customers can register with their name, email, and allergy information. The system validates important inputs such as email format and prevents duplicate registration with the same email. After logging in, customers can browse the product catalog, select products by displayed list number, add products to their carts, remove cart items, review allergy warnings, choose a payment method, and create orders. Admin users can manage products, update stock information, disable products, list customers, view orders, and update order statuses according to valid lifecycle rules. These operations make the project suitable for analysis, design, UML modeling, design pattern discussion, SOLID evaluation, and testing.

1.1 Project Objectives

The main objective of this project is to design and implement a medium-scale Java application that demonstrates object-oriented analysis and design principles. The system is not intended to be a production-ready commercial grocery platform. Instead, it focuses on clear software structure, layered architecture, UML modeling, design pattern usage, SOLID principles, and maintainable Java implementation.

The project also aims to solve a meaningful business problem by extending a standard grocery shopping flow with allergy awareness. Customers can register with allergy

information, browse products, receive warnings when a product contains risky allergens, manage their cart, and complete checkout through simulated payment methods. Admin users can manage products, stock information, customers, orders, and order status transitions.

Another objective is to show how analysis and design decisions are reflected in the implementation. The use cases, UML diagrams, layered package structure, domain classes, service classes, repositories, design patterns, and testing approach are all connected to the final Java code.

2. Problem Definition

Online grocery shopping systems are commonly designed around basic e-commerce operations such as product listing, cart management, stock control, payment, and order tracking. These operations are necessary for a functional shopping system, but they are not always sufficient for every type of customer. In grocery shopping, products may contain allergens such as milk, egg, gluten, nuts, soy, or seafood. For customers with food allergies, buying an unsuitable product may create a health risk. Therefore, a grocery system should not only process orders, but should also help customers identify products that may be unsafe for them.

The main problem addressed in this project is the lack of allergy-aware decision support in a standard grocery shopping process. In many simple shopping systems, a product is treated only as an item with a name, category, price, and stock quantity. However, in a grocery context, allergen information is also an important part of the product data. Similarly, a customer is usually represented only with basic identity information, but in this project the customer also has an allergy profile. If the system does not compare the product allergens with the customer's allergy profile, the customer may accidentally add a risky product to the cart and complete an order without being properly warned.

The Allergy Aware Grocery System is designed to solve this problem by integrating allergy checks into the shopping flow. When a customer registers, the system allows the customer to define an allergy profile. When the customer selects a product and tries to add it to the cart, the system checks the product's allergen list against the customer's allergy profile. If there is a conflict, the system shows a warning and asks the customer to confirm the risk. This means that risky products are not silently added to the cart. The allergy warning is also kept as part of the cart item information, so that the risk remains visible during later stages of shopping.

Another part of the problem is that checkout must be controlled, not only performed. A weakly designed system may allow checkout even when the cart is empty, the selected product is no longer active, the requested quantity is greater than the available stock, or the payment cannot be completed. In this project, checkout is treated as a business process that requires several validations. The cart must contain at least one item. Each product must still be active. Stock must be sufficient. Payment must be processed successfully. If these checks are not handled in a structured way, the system may create incorrect orders, reduce stock incorrectly, or accept invalid purchase attempts.

The system also addresses the need for administrative control. A grocery system cannot be managed only from the customer side. Admin users must be able to view and manage products, update stock quantities, disable products that should no longer be sold, view customer information, view orders, and update order statuses. Without admin operations, the system would represent only the customer-facing part of e-commerce and would not be complete enough as an order processing system. Therefore, the project includes both customer and admin flows.

In summary, the problem can be defined as designing and implementing a grocery order processing system that supports standard e-commerce operations while also protecting allergy-sensitive customers through warning and confirmation mechanisms. The system must allow customers to shop safely, allow admins to manage products and orders, and demonstrate a maintainable object-oriented Java design. The Allergy Aware Grocery System solves this problem by combining allergy-aware product checking, cart and checkout validation, order lifecycle management, simulated payment processing, and layered software architecture.

3. Requirements

This section defines the functional and non-functional requirements of the Allergy Aware Grocery System. Functional requirements describe what the system must do from the perspective of customers, admins, and business processes. Non-functional requirements describe the quality expectations of the system, such as maintainability, performance, security, reliability, scalability, and usability.

The system is designed as a console-based Java application. Therefore, the requirements focus on business logic, object-oriented design, layered architecture, and correct system behavior rather than graphical interface design or database implementation.

3.1 Functional Requirements

Functional requirements define the main operations that the system must support. In this project, the system has two main actor groups: **Customer** and **Admin**. Customers interact with the system to register, login, browse products, manage their cart, receive allergy warnings, and place orders. Admin users interact with the system to manage products, view customers, view orders, and update order statuses.

FR-01: Customer Registration

The system shall allow a new customer to register by entering required personal information such as name and email address.

During registration, the system shall validate that the customer name is not empty. The system shall also validate the email address format. An email address must contain a valid structure such as a local part, @ symbol, domain name, and domain extension. Invalid email inputs shall not be accepted.

The system shall prevent duplicate registration with the same email address. If a customer tries to register with an email address that already exists in the system, the system shall display an error message and guide the customer to login instead.

When registration is successful, the system shall create a customer account, assign a unique customer ID, create an allergy profile for the customer, create an empty cart for the customer, and automatically login the customer.

FR-02: Customer Login

The system shall allow registered customers to login by entering their email address.

The system shall validate the email format before searching for a customer. If the email format is invalid, the system shall display an error message. If the email format is valid but no customer exists with that email, the system shall inform the user that registration is required.

When login is successful, the system shall display the customer's wallet balance and allergy profile information.

FR-03: Allergy Profile Creation

The system shall allow customers to define their allergy profile during registration.

The allergy profile shall consist of selected allergens from the predefined allergen list. A customer may select one allergen, multiple allergens, or no allergens. The system shall store the selected allergens as part of the customer profile.

The allergy profile shall later be used by the allergy checking mechanism when the customer tries to add products to the cart or complete checkout.

FR-04: Product Catalog Browsing

The system shall allow customers to browse the active product catalog.

The product catalog shall display product information such as product number in the displayed list, product name, category, price, stock quantity, and declared allergens. Customers shall select products by entering the displayed list number instead of typing product codes manually. This reduces user mistakes and makes the console interface easier to use.

Disabled products shall not be displayed to customers as available products. However, admin users shall be able to view both active and disabled products for management purposes.

FR-05: Product Selection and Cart Addition

The system shall allow logged-in customers to add products to their cart.

Before adding a product, the system shall require the customer to select a product from the displayed product list and enter a valid quantity. The quantity must be greater than zero. The system shall reject zero, negative, non-numeric, or invalid quantity inputs.

The system shall check whether the selected product exists, whether it is active, and whether the requested quantity is available in stock. If any of these validations fail, the product shall not be added to the cart.

FR-06: Allergy Warning Before Cart Addition

The system shall check the selected product's allergens against the logged-in customer's allergy profile before the product is added to the cart.

If the product contains one or more allergens that match the customer's allergy profile, the system shall display a clear allergy warning. The warning shall include the selected product name, matched allergens, and customer allergy profile information.

The system shall ask the customer whether they still want to add the product despite the warning. If the customer does not confirm the risk, the product shall not be added to the cart. If the customer confirms the risk, the product shall be added to the cart with allergy warning information.

FR-07: Cart Viewing

The system shall allow logged-in customers to view their cart.

The cart view shall display the products currently added to the cart, quantities, unit prices, line totals, and total cart amount. If a cart item has an allergy warning, the warning shall remain visible in the cart.

If the customer's cart is empty, the system shall display an appropriate message.

FR-08: Cart Item Removal

The system shall allow logged-in customers to remove items from their cart.

The system shall display the cart items with list numbers. The customer shall select the item to remove by entering the displayed cart item number. If the customer selects 0, the remove operation shall be cancelled.

The system shall validate the selected item number. Invalid item numbers shall not remove any product from the cart.

FR-09: Checkout Process

The system shall allow logged-in customers to checkout when their cart is not empty.

Before checkout is completed, the system shall validate the cart. The cart must contain at least one item. Each product in the cart must still be active. Each product must have enough stock for the requested quantity.

If the cart contains allergy warnings, the system shall display the warnings again during checkout and ask for final confirmation. If the customer does not confirm the warnings, checkout shall be cancelled.

If the cart is valid and allergy warnings are confirmed when necessary, the system shall continue with payment processing.

FR-10: Payment Method Selection

The system shall allow the customer to select a payment method during checkout.

The implemented payment methods are credit card, debit card, and wallet balance. Cash on delivery is not included in the project scope.

For wallet payment, the system shall check whether the customer has enough wallet balance. If the balance is insufficient, the payment shall be declined.

For card payments, the system shall validate required card information such as card holder name, card number, expiry month, expiry year, and CVV. The card number must contain exactly 16 digits. The CVV must contain exactly 3 digits. The expiry month must be between 1 and 12. Expired cards shall not be accepted.

FR-11: Payment Processing

The system shall process payment requests through a payment service.

For wallet balance payment, the system shall deduct the order amount from the customer's wallet if the balance is sufficient.

For card payments, the system shall use a payment gateway abstraction. The application shall communicate with the payment gateway through an adapter so that the payment logic is separated from the rest of the checkout process.

If payment is approved, the checkout process shall continue. If payment fails or is declined, the order shall not be completed successfully.

FR-12: Order Creation

The system shall create an order from the customer's cart after successful checkout validation and payment approval.

The created order shall contain a unique order ID, customer ID, order items, item quantities, item prices, total amount, and initial order status.

After an order is created and payment is approved, the order status shall move from CREATED to PAID.

FR-13: Stock Update After Checkout

The system shall decrease product stock quantities after successful checkout.

Stock shall only be decreased after payment is approved and the order is successfully moved to the paid state. If payment fails, product stock shall not be decreased.

The system shall prevent checkout if any cart item quantity is greater than the available product stock.

FR-14: Cart Clearing After Successful Checkout

The system shall clear the customer's cart after successful checkout.

Cart clearing shall only happen after the order is created, payment is approved, order status is updated, and product stock is decreased.

If checkout fails at any stage, the cart shall not be cleared automatically. This allows the customer to review and correct the problem.

FR-15: Customer Order Viewing

The system shall allow customers to view their own orders.

When a customer selects the order viewing option, the system shall display only the orders belonging to the logged-in customer. The customer shall be able to see order summary information such as order ID, total amount, and current order status.

FR-16: Admin Authentication

The system shall provide a separate admin mode.

Before accessing admin operations, the system shall require an admin password. If the entered password is incorrect, the system shall deny access and return to the main menu.

Admin authentication is simple in this project because the main focus is object-oriented design and business logic rather than advanced security infrastructure.

FR-17: Admin Product Viewing

The system shall allow admin users to view all products.

Unlike the customer product catalog, the admin product view shall include both active and disabled products. This allows the admin to monitor the full product list and manage product availability.

FR-18: Admin Product Addition

The system shall allow admin users to add new products.

When adding a product, the admin shall enter the product name, category, price, stock quantity, and declared allergens. The system shall validate that the product name is not empty, the price is greater than zero, and the stock quantity is not negative.

After successful product creation, the system shall assign a unique product ID and save the product in the product repository.

FR-19: Admin Stock Update

The system shall allow admin users to update product stock.

The admin shall select a product from the displayed product list and enter a new stock quantity. The new stock quantity must not be negative. If the entered stock value is invalid, the system shall reject it.

FR-20: Admin Product Disabling

The system shall allow admin users to disable products.

When a product is disabled, it shall no longer be displayed in the customer product catalog. However, it shall remain visible to admin users. This is important because disabling a product preserves product history while preventing further customer purchases.

The system shall ask for confirmation before disabling a product.

FR-21: Admin Order Viewing

The system shall allow admin users to view all orders in the system.

Admin users shall be able to see order summaries and select an order to view its details. Order details shall include customer ID, ordered items, quantities, total amount, and current order status.

FR-22: Admin Order Status Update

The system shall allow admin users to update order statuses according to valid lifecycle rules.

The order lifecycle shall follow controlled transitions. For example, a created order may move to paid or cancelled. A paid order may move to preparing or cancelled. A preparing order may move to shipped or cancelled. A shipped order may move to delivered. Delivered and cancelled orders shall be final states.

The system shall not allow invalid status transitions. If an admin tries to move an order to an unavailable status, the system shall reject the operation.

FR-23: Order Notification and Logging

The system shall notify observers when an order status changes.

The system shall use observer objects to create customer notification messages and order log entries. When the order status changes successfully, notification and log information shall be added to the order.

This requirement helps the system demonstrate event-based behavior without implementing a real email or notification service.

FR-24: Input Validation

The system shall validate user input throughout the console application.

The system shall handle invalid menu selections, non-numeric input where numbers are expected, empty text input, invalid email addresses, invalid money amounts, invalid stock values, invalid quantities, invalid card numbers, invalid expiry dates, and invalid yes/no answers.

Invalid inputs shall not crash the application. Instead, the system shall display an explanatory message and ask the user to enter the value again.

FR-25: In-Memory Data Storage

The system shall store customers, products, carts, and orders using in-memory repositories.

Repository interfaces shall be defined separately from their concrete implementations. The application services shall depend on repository abstractions, while the infrastructure layer shall provide in-memory implementations.

This supports layered architecture and dependency inversion while keeping the project scope suitable for a console-based term project.

3.2 Non-Functional Requirements

Non-functional requirements define how well the system should perform and how the system should be structured. Since this project is mainly evaluated from an object-oriented design perspective, non-functional requirements are especially important.

NFR-01: Maintainability

The system shall be easy to maintain by separating responsibilities between classes and layers.

Presentation classes shall handle console menus and user interaction. Application service classes shall coordinate use cases and business operations. Domain classes shall represent core business concepts and rules. Infrastructure classes shall handle technical details such as in-memory repositories, seed data, and payment gateway adaptation.

This separation reduces the risk of having one large class responsible for many unrelated tasks.

NFR-02: Layered Architecture Compliance

The system shall follow a layered architecture.

The main layers of the system are presentation, application/service, domain, and infrastructure. The presentation layer shall not directly manage repositories. The domain layer shall not depend on console input or infrastructure details. Infrastructure classes shall implement repository and gateway abstractions.

This layered design allows the system to be extended more easily in the future. For example, the console interface could later be replaced with a graphical interface, or the in-memory repositories could be replaced with database-backed repositories.

NFR-03: Usability

The system shall provide understandable console menus and clear user messages.

Customers and admins shall select options using displayed menu numbers. Products and cart items shall also be selected by displayed list numbers instead of manually typing complex product IDs. This improves usability and reduces typing errors.

The system shall display meaningful success and failure messages so that users can understand what happened and what they need to do next.

NFR-04: Reliability

The system shall behave consistently during normal and invalid operations.

Invalid input shall not terminate the program. Failed validation shall not corrupt system data. For example, if payment fails, stock shall not be decreased and the cart shall not be cleared. If an allergy warning is not confirmed, the risky product shall not be added to the cart. If checkout is cancelled, the system shall keep the cart available for correction.

NFR-05: Data Consistency

The system shall keep customer, cart, product, and order data consistent during runtime.

A customer shall have one cart after registration. Product stock shall only be decreased after successful payment. Disabled products shall not be available for new customer purchases. Orders shall follow valid status transitions. Invalid transitions shall not change the order state.

NFR-06: Security

The system shall include basic security controls suitable for the project scope.

Admin operations shall be protected by an admin password. Customer login shall require a valid registered email address. The system shall validate sensitive payment inputs such as card number and CVV format.

However, advanced security features such as password hashing, encrypted storage, real authentication tokens, and secure payment integration are outside the current project scope because the application uses a console interface and in-memory storage.

NFR-07: Performance

The system shall respond quickly for typical console-based operations.

Since the project uses in-memory repositories, operations such as listing products, finding customers by email, viewing cart contents, and viewing orders are expected to run efficiently for small and medium-sized test data.

The system is not designed for high-volume commercial traffic. However, the use of separated repositories and services makes future optimization possible.

NFR-08: Scalability

The system shall be designed in a way that allows future growth.

Although the current version uses in-memory repositories, repository interfaces allow the infrastructure layer to be replaced with database-backed implementations. Similarly, the payment gateway abstraction allows different payment providers to be added later. The allergy checking strategy can also be replaced or extended with more advanced allergy rules.

NFR-09: Extensibility

The system shall allow new features to be added with minimum changes to existing code.

For example, new product categories, new allergens, new payment methods, new notification observers, or new order states can be added by extending the relevant parts of the system. The use of design patterns such as Strategy, Adapter, Facade, Observer, State, Builder, Factory Method, and Singleton supports this extensibility.

NFR-10: Testability

The system shall be structured so that important business logic can be tested independently.

Application services such as CustomerService, CartService, PaymentService, OrderService, and CheckoutFacade can be tested without directly depending on console input. Repository interfaces and in-memory implementations make it possible to create controlled test data.

The console layer is less suitable for automated unit testing, but most business logic is placed outside the console layer to improve testability.

NFR-11: Clean Code

The system shall use clear class names, method names, and package organization.

Each class shall have a clear responsibility. Methods shall be understandable and should not mix unrelated concerns. For example, customer registration logic, cart logic, payment logic, order logic, and admin logic are placed in separate service classes.

The code shall avoid unnecessary complexity while still demonstrating object-oriented design principles.

NFR-12: SOLID Compatibility

The system shall aim to follow SOLID principles.

Classes should have focused responsibilities. Services should depend on abstractions where appropriate. New behavior should be added through extension rather than unnecessary modification of existing classes. Interfaces should represent focused contracts. Domain abstractions should not depend on low-level infrastructure details.

The SOLID evaluation section of the report will discuss the compliance level of the project, possible violations, and suggested improvements.

NFR-13: Availability Within Runtime

The system shall remain available while the console application is running.

Because the application does not use a persistent database, all data is stored only during program execution. When the program closes, runtime-created customers, carts, products, and orders are lost. This is acceptable for the current project scope but should be considered a limitation.

NFR-14: Portability

The system shall be runnable on any environment that supports Java compilation and execution.

The project does not require an external database, GUI framework, web server, or third-party payment service. It can be compiled and executed from the command line using standard Java tools.

NFR-15: Error Handling

The system shall handle expected error cases gracefully.

Examples of expected error cases include invalid email format, duplicate customer email, empty cart checkout, insufficient stock, inactive product selection, unconfirmed allergy warning, insufficient wallet balance, expired card, invalid card number, invalid CVV, and invalid order status transition.

The system shall return operation results with meaningful messages instead of allowing unexpected crashes in normal user flows.

3.3 Use Cases

3.3.1 Use Case Descriptions

Use cases describe how users interact with the Allergy Aware Grocery System to achieve specific goals. In this project, the system has two main actors: **Customer** and **Admin**. The customer uses the system to register, login, define allergy information, browse products, manage the cart, complete checkout, make payment, and view personal orders. The admin uses the system to manage products, update stock, disable products, view customers, view orders, and update order statuses.

The following four use cases represent the most important business scenarios of the system. These scenarios are selected because they cover both customer-side and admin-side operations. They also include the main object-oriented design features of the project such as allergy checking, cart management, checkout coordination, payment processing, order lifecycle control, and admin product management.

The use case diagram will be presented later in the UML Diagrams section. In this section, only the written use case descriptions are provided.

3.3.2 Actors

The first actor is the **Customer**. A customer is a registered user who wants to buy grocery products from the system. The customer can register, login, create an allergy profile, browse products, add products to the cart, remove cart items, complete checkout, make payment, and view previous orders. Since the system is allergy-aware, the customer's allergy profile is an important part of the shopping process.

The second actor is the **Admin**. The admin is responsible for managing the operational side of the grocery system. The admin can view all products, add new products, update stock quantities, disable products, view customers, view orders, view order details, and update order statuses. Admin operations are protected with a simple password mechanism in the console application.

3.3.3 Use Case Summary

The main use cases discussed in this report are listed below.

UC-01: Register and Create Allergy Profile
This use case explains how a new customer creates an account and defines allergy information.

UC-02: Browse Products and Add Product to Cart with Allergy Check
This use case explains how a customer browses available products and adds a product to the cart after allergy and stock validation.

UC-03: Checkout and Payment Processing
This use case explains how the customer completes an order by validating the cart, confirming allergy warnings, selecting a payment method, processing payment, creating an order, updating stock, and clearing the cart.

UC-04: Admin Manages Products and Order Statuses
This use case explains how the admin manages products, stock quantities, disabled products, orders, and order status transitions.

3.3.3.1 UC-01: Register and Create Allergy Profile

Use Case ID: UC-01
Use Case Name: Register and Create Allergy Profile
Primary Actor: Customer
Main Goal: To create a new customer account and define the customer's allergy profile.
Trigger: The customer selects the registration option from the customer menu.
Related Classes: CustomerMenu, CustomerService, CustomerBuilder, CustomerRepository, CartRepository, Customer, AllergyProfile, Cart

Description

This use case starts when a new customer wants to use the system for shopping. Since the Allergy Aware Grocery System provides customer-specific allergy warnings, each customer must have an account and an allergy profile. Registration is therefore not only used for storing basic customer information, but also for preparing the system to give personalized allergy warnings during shopping.

The customer enters their name and email address. The system validates that the name is not empty and that the email address has a valid format. The system also checks whether the same email address already exists in the customer repository. This prevents duplicate registration and keeps customer records consistent.

After the basic customer information is validated, the system asks the customer to select allergens from the predefined allergen list. The customer may select one allergen, multiple allergens, or no allergen. Selecting no allergen is accepted because

not every customer has a food allergy. The selected allergens are stored inside the customer's allergy profile.

When all registration information is valid, the system creates a new Customer object. In the implementation, the customer object is created with the help of CustomerBuilder. After the customer is saved, the system also creates an empty cart for that customer. Finally, the new customer is automatically logged in.

Preconditions

The application must be running.
The customer must not already be logged in.
The entered email address must not already belong to another registered customer.
The customer menu must be accessible.

Main Scenario

The customer opens the customer menu and selects the registration option. The system asks the customer to enter a name. The customer enters a valid non-empty name. Then, the system asks for an email address. The customer enters an email address, and the system validates its format.

After the email format is accepted, the system checks whether the email is already registered. If the email is unique, the system displays the available allergen options. The customer selects allergens according to their allergy profile. The system creates the allergy profile and uses the entered information to create a new customer.

The new customer is saved in the customer repository. Then, an empty cart is created for the customer and saved in the cart repository. After successful registration, the customer becomes logged in and the system displays the customer's wallet balance and allergy profile.

Alternative and Exception Flows

If the customer enters an empty name, the system rejects the input and asks for a valid name.
If the customer enters an invalid email address, the system displays an error message and asks for the email again.
If the email address is already registered, the system does not create a new account and informs the customer that the email already exists.
If the customer does not select any allergens, the system creates an empty allergy profile and continues registration normally.

Postconditions

A new customer account is created.
The customer has an allergy profile.
An empty cart is created for the customer.
The customer is automatically logged in.
The customer can continue with product browsing and shopping operations.

3.3.3.2 UC-02: Browse Products and Add Product to Cart with Allergy Check

Use Case ID: UC-02
Use Case Name: Browse Products and Add Product to Cart with Allergy Check
Primary Actor: Customer
Main Goal: To allow the customer to select a product and add it to the cart after stock and allergy checks.
Trigger: The customer selects the browse products or add product to cart option.
Related Classes: CustomerMenu, CatalogService, CartService, AllergyCheckService, AllergyCheckStrategy, WarningConfirmationAllergyStrategy, ProductRepository, CartRepository, Product, Cart, CartItem, AllergyCheckResult

Description

This use case represents one of the core customer operations of the system. The customer browses the product catalog and selects a product to add to the cart. Since this is an allergy-aware grocery system, adding a product to the cart is not handled as a simple product insertion operation. Before the product is added, the system checks whether the product contains allergens that conflict with the customer's allergy profile.

The system displays only active products to customers. Each product is shown with a list number, product name, category, price, stock quantity, and declared allergens. The customer selects the product by entering the displayed list number. This design is more user-friendly than asking the customer to type product codes manually.

After selecting a product, the customer enters the desired quantity. The system validates that the quantity is a positive number. Then, the system checks whether the product is active and whether the requested quantity is available in stock. If these checks are successful, the system performs allergy checking.

The allergy checking operation compares the allergens of the selected product with the customer's allergy profile. If no matching allergen is found, the product is added to the

cart normally. If a matching allergen exists, the system displays a warning and asks the customer whether they still want to add the product. If the customer confirms the risk, the product is added to the cart with allergy warning information. If the customer refuses, the product is not added.

Preconditions

The customer must be registered.
The customer must be logged in.
The product catalog must contain at least one active product.
The customer must have an existing cart.

Main Scenario

The customer selects the product browsing option from the customer menu. The system retrieves active products through CatalogService and displays them with list numbers. The customer reviews the product list and selects a product by entering the displayed number.

The system asks the customer to enter the quantity. The customer enters a positive quantity. The system validates the selected product, checks whether it is active, and verifies that there is enough stock.

After product and stock validation, the system calls the allergy checking logic. The allergy checking service compares the customer's allergy profile with the selected product's allergens. If there is no allergy conflict, the product is added to the cart. The system displays a success message.

If there is an allergy conflict, the system shows an allergy warning. The warning includes the selected product and matched allergens. The customer is asked whether they want to add the product despite the risk. If the customer confirms, the product is added to the cart with warning information. If the customer does not confirm, the operation is cancelled.

Alternative and Exception Flows

If the customer selects an invalid product number, the system rejects the selection and no product is added.
If the customer enters zero, a negative number, or invalid quantity input, the system asks for a valid quantity.
If the selected product is inactive, the system does not allow the product to be added to the cart.

If the requested quantity is greater than the available stock, the system displays an insufficient stock message.
If the product contains allergens matching the customer's allergy profile, the system displays an allergy warning.
If the customer does not confirm the allergy warning, the product is not added to the cart.

Postconditions

If all validations are successful, the product is added to the customer's cart.
If the product has an allergy risk and the customer confirms it, the cart item stores allergy warning information.
If validation fails, the customer's cart remains unchanged.

3.3.3.3 UC-03: Checkout and Payment Processing

Use Case ID: UC-03
Use Case Name: Checkout and Payment Processing
Primary Actor: Customer
Main Goal: To complete the customer's order by validating the cart, confirming allergy warnings, processing payment, creating an order, updating stock, and clearing the cart.
Trigger: The customer selects the checkout option from the customer menu.
Related Classes: CustomerMenu, CheckoutFacade, CartService, OrderService, PaymentService, PaymentGateway, PaymentGatewayAdapter, ExternalPaymentGateway, Cart, CartItem, Order, OrderItem, PaymentRequest, PaymentResult, CheckoutResult, Money

Description

This use case describes the checkout process of the Allergy Aware Grocery System. Checkout is the most important customer-side business process because it brings together many operations. The system must validate the cart, remind the customer about allergy warnings, collect payment information, process the payment, create the order, update product stock, and clear the cart after successful completion.

In the implementation, checkout is coordinated by CheckoutFacade. This is important because checkout is not a single simple operation. If the presentation layer handled every checkout step directly, the console menu would become too complex and would contain too much business logic. Instead, the facade provides a single higher-level checkout operation and coordinates the related services.

The checkout process begins when the customer selects the checkout option. The system retrieves the customer's cart and validates it. The cart must not be empty. All products in the cart must still be active. The requested quantities must be available in stock. If any of these conditions fail, checkout is cancelled.

If the cart contains products with allergy warnings, the system displays these warnings again before payment. This is necessary because the customer may have confirmed the allergy warning earlier while adding the product to the cart. Before finalizing the purchase, the system asks the customer to confirm the risk again.

After the cart is validated and allergy warnings are confirmed, the customer selects a payment method. The system supports credit card, debit card, and wallet balance payments. Cash on delivery is not part of the project scope. Payment information is validated according to the selected payment method. If payment is approved, the order is created and completed. If payment fails, the order is not completed successfully, stock is not decreased, and the cart is not cleared.

Preconditions

The customer must be logged in.
The customer must have an existing cart.
The cart must contain at least one item.
Products in the cart must be active.
Product stock must be sufficient for all cart items.

Main Scenario

The customer selects checkout from the customer menu. The system retrieves the customer's cart using CartService. The system validates that the cart is not empty. Then, it checks whether all products in the cart are still active and whether enough stock is available.

If there are allergy warnings in the cart, the system displays the risky products and matched allergens. The customer confirms that they understand the warning and want to continue checkout.

The system displays payment options. The customer selects a payment method and enters the required payment information. The system creates a PaymentRequest.

The checkout request is sent to CheckoutFacade. The facade coordinates the operation by using CartService, OrderService, and PaymentService. An order is created from the cart. The payment is processed. If payment is successful, the order status is

updated to paid. Product stock quantities are decreased, and the customer's cart is cleared.

Finally, the system displays a successful checkout message and shows the order details.

Alternative and Exception Flows

If the cart is empty, the system cancels checkout and displays an appropriate message.
If a product in the cart is inactive, checkout is cancelled.
If stock is insufficient for one or more cart items, checkout is cancelled.
If the cart contains allergy warnings and the customer does not confirm them, checkout is cancelled.
If the customer cancels payment method selection, checkout is cancelled.
If the customer selects wallet payment but has insufficient wallet balance, payment is declined.
If the customer enters invalid card information, the system rejects the payment information.
If the external payment gateway simulation declines the payment, the order is not completed successfully.

Postconditions

If checkout is successful, a new order is created.
The order status becomes paid.
Product stock quantities are decreased.
The customer's cart is cleared.
If checkout fails, stock is not decreased and the cart is not cleared.

3.3.3.4 UC-04: Admin Manages Products and Order Statuses

Use Case ID: UC-04
Use Case Name: Admin Manages Products and Order Statuses
Primary Actor: Admin
Main Goal: To manage product information, stock quantities, disabled products, customer records, orders, and order status transitions.
Trigger: The admin selects admin mode and enters the correct admin password.
Related Classes: AdminMenu, AdminService, ProductRepository, OrderRepository, CustomerRepository, ProductFactory, Product, Order, OrderState, CreatedState,

PaidState, PreparingState, ShippedState, DeliveredState, CancelledState, OrderObserver, CustomerNotificationObserver, OrderLogObserver

Description

This use case represents the main admin-side operations of the system. A grocery shopping system cannot be complete with only customer-side functions. The product catalog must be managed, stock quantities must be updated, unavailable products must be disabled, customer information must be viewable, and customer orders must be managed after checkout.

The admin flow starts with a simple password check. If the admin enters the correct password, the system displays the admin menu. If the password is incorrect, access is denied and the system returns to the main menu.

After login, the admin can view all products. Unlike the customer catalog, the admin product list includes both active and disabled products. The admin can add a new product by entering the product name, category, price, stock quantity, and allergen information. The system validates this information before creating the product. Product creation is supported by ProductFactory.

The admin can also update stock quantities. The system displays the product list and asks the admin to select a product. The admin enters the new stock value, and the system validates that the value is not negative. If valid, the product stock is updated.

Another product management operation is disabling a product. When a product is disabled, it remains in the system but is no longer shown to customers as an available product. This is better than deleting the product because it preserves product information while preventing new purchases.

The admin also manages orders. The admin can view all orders, inspect order details, and update order statuses. Order status changes are controlled by the State pattern. The system does not allow random status changes. For example, an order can move from paid to preparing, from preparing to shipped, and from shipped to delivered. Delivered and cancelled orders are final states.

When an order status is updated successfully, observer objects are notified. The customer notification observer adds a notification message, and the order log observer adds a log entry. This allows the system to demonstrate observer-based behavior without implementing a real notification service.

Preconditions

The application must be running.
The admin must select admin mode.
The admin must enter the correct admin password.
For product operations, the product repository must be accessible.
For order status updates, at least one order should exist in the system.

Main Scenario

The admin selects admin mode from the main menu. The system asks for the admin password. The admin enters the correct password, and the system displays the admin menu.

For product management, the admin can view the full product list. The admin may add a new product by entering product name, category, price, stock quantity, and allergens. The system validates the entered values and creates the product using ProductFactory. The product is saved in the product repository.

The admin may also update product stock. The system displays the product list, and the admin selects a product by list number. The admin enters the new stock quantity. If the value is valid, the system updates the stock.

The admin may also disable a product. The system asks for confirmation before disabling. If the admin confirms, the selected product becomes inactive and is hidden from customer product browsing.

For order management, the admin can list all orders and view order details. The admin can select an order and choose a new status from the available next statuses. The system validates the requested transition using the order state structure. If the transition is valid, the order status is updated and observers are notified.

Alternative and Exception Flows

If the admin enters the wrong password, access is denied and the admin menu is not displayed.
If the admin enters an invalid product name, invalid price, or invalid stock quantity, the system rejects the product operation.
If the admin cancels product disabling, the product remains active.
If the selected product is already disabled, the system informs the admin.
If no orders exist, the system displays an empty order list.
If an order is already in a final state, the system does not allow further status changes.

If the admin tries to perform an invalid status transition, the system rejects the operation and the order status remains unchanged.

Postconditions

Product changes are saved in the product repository.
New active products become visible to customers.
Disabled products are hidden from the customer catalog.
Stock changes are reflected in future shopping operations.
Valid order status changes are saved.
Order notification and log entries are created after successful status updates.
Invalid operations do not change system data.

4.Architectural Design

4.1 Architectural Overview

The Allergy Aware Grocery System is designed with a layered architecture. The main goal of this architecture is to separate user interaction, application logic, domain rules, and infrastructure details from each other. This separation makes the system easier to understand, test, maintain, and extend.

The project is divided into four main layers:

- Presentation Layer

- Application / Service Layer

- Domain Layer

- Infrastructure Layer

The presentation layer is responsible for console-based user interaction. The application/service layer manages use case logic such as registration, cart operations, checkout, payment, and admin operations. The domain layer contains the main business objects, business rules, interfaces, and design pattern participants. The infrastructure layer contains technical implementations such as in-memory repositories, the payment adapter, product seeding, and application configuration.

This architecture was selected because the project is not only expected to work as a console application, but also to demonstrate object-oriented design quality. If all logic

were written inside menu classes, the system would become difficult to maintain. By using layers, each part of the system has a clearer responsibility.

4.2 Presentation Layer

The presentation layer contains the console classes that interact directly with the user. In this project, the main presentation classes are:

ConsoleApp
CustomerMenu
AdminMenu
InputReader
ConsolePrinter

ConsoleApp starts the main menu flow. CustomerMenu handles customer-related actions such as registration, login, browsing products, adding products to cart, viewing cart, checkout, and viewing orders. AdminMenu handles admin operations such as adding products, updating stock, disabling products, viewing customers, viewing orders, and updating order statuses.

InputReader is responsible for reading and validating console inputs such as numbers, e-mails, money values, and yes/no answers. ConsolePrinter is responsible for displaying products, carts, orders, customers, and payment results.

The presentation layer does not directly manage business rules. For example, it does not directly decide whether a product is safe for an allergic customer or whether an order status transition is valid. Instead, it calls application services and displays the returned results to the user.

4.3 Application / Service Layer

The application/service layer contains the main use case coordination logic. The important service classes are:

CustomerService
CatalogService
CartService
AllergyCheckService
OrderService
PaymentService
AdminService
CheckoutFacade

CustomerService handles customer registration and login. It validates customer information, checks whether the e-mail address is already registered, creates the allergy profile, builds the customer object, saves the customer, and creates an empty cart.

CatalogService provides product listing and product search operations.

CartService manages cart operations. It retrieves the customer cart, validates product availability and stock, performs allergy checking before adding a product, removes cart items, validates the cart before checkout, decreases stock after successful checkout, and clears the cart.

AllergyCheckService delegates allergy checking behavior to the allergy checking strategy. This keeps allergy checking rules separate from cart logic.

OrderService creates orders from carts and manages order status transitions. It also attaches observers to orders so that notifications and log messages can be created when order status changes.

PaymentService validates and processes payments. Wallet payments are handled directly inside the service, while credit card and debit card payments are processed through the payment gateway abstraction.

AdminService manages admin operations such as product creation, stock update, product disabling, viewing customers and orders, and updating order statuses.

CheckoutFacade simplifies the checkout process for the presentation layer. Checkout requires cart validation, allergy warning confirmation, order creation, payment processing, order status update, stock decrease, and cart clearing. Instead of forcing CustomerMenu to coordinate all these services separately, CheckoutFacade provides a single checkout operation.

4.4 Domain Layer

The domain layer contains the core business model of the system. The main domain model classes are:

Customer
AllergyProfile
Product
Cart
CartItem
Order

OrderItem
Money
AllergyCheckResult
PaymentRequest
PaymentResult
CheckoutResult
OperationResult

Customer represents a registered customer and contains the customer's allergy profile and wallet balance. AllergyProfile stores the customer's allergens. Product represents a grocery product with category, price, stock, active status, and allergen information.

Cart and CartItem represent the customer's shopping cart. A cart item can store allergy warning information if the customer adds a risky product after confirmation.

Order and OrderItem represent completed checkout data. Order uses the State pattern to control valid status transitions. Money is used as a value object for price and total calculations.

The domain layer also contains important interfaces and pattern-related classes:

Repository interfaces
AllergyCheckStrategy
OrderState
OrderObserver
PaymentGateway
CustomerBuilder
ProductFactory

These interfaces and classes allow the application to depend on abstractions instead of concrete technical details.

4.5 Infrastructure Layer

The infrastructure layer contains the technical implementations of the abstractions defined in the domain layer. In this project, the infrastructure layer includes:

InMemoryCustomerRepository
InMemoryProductRepository
InMemoryCartRepository
InMemoryOrderRepository
PaymentGatewayAdapter
ExternalPaymentGateway

ExternalPaymentResponse
ApplicationContext
ProductSeeder

The repository implementations store data in memory. This is suitable for the project because the focus is object-oriented design and architecture, not database implementation. The service layer depends on repository interfaces, so these in-memory repositories could be replaced with database-based repositories in the future.

PaymentGatewayAdapter adapts the simulated external payment gateway to the internal PaymentGateway interface. This keeps external payment details outside the service layer.

ApplicationContext is responsible for creating objects and connecting dependencies manually. It creates repositories, services, strategies, factories, the payment adapter, the checkout facade, and the console application. This works as manual dependency injection.

ProductSeeder creates initial sample products and saves them into the product repository when the application starts.

4.6 Dependency Direction

The dependency direction of the system is designed to support layered architecture.

The general dependency flow is:

Presentation Layer -> Application / Service Layer -> Domain Layer

Infrastructure Layer implements domain abstractions.

The presentation layer depends on application services because it needs to call use cases. The application layer depends on domain models and domain interfaces. The infrastructure layer depends on domain interfaces because it provides concrete implementations for repositories and payment gateway integration.

A key point is that the domain layer does not depend on the presentation layer or infrastructure layer. This protects the business logic from technical details. For example, Order, Cart, Product, and Customer do not know whether the application is using a console interface, a web interface, in-memory repositories, or a database.

4.7 Manual Dependency Injection

The project uses manual dependency injection through ApplicationContext. Instead of creating dependencies inside service classes, dependencies are created once and passed through constructors.

For example, CartService receives CartRepository, ProductRepository, and AllergyCheckService. PaymentService receives a PaymentGateway. CheckoutFacade receives CartService, OrderService, and PaymentService.

This design improves maintainability and testability. A service class does not need to know which concrete repository or gateway implementation is used. In the current project, in-memory repositories are used. In a future version, they could be replaced with database repositories without changing most of the service logic.

4.8 Architectural Benefits

The selected architecture provides several benefits.

First, it improves separation of concerns. Console input/output, business workflows, domain rules, and infrastructure details are not mixed in one class.

Second, it improves maintainability. If product creation rules change, ProductFactory can be updated. If allergy checking rules change, a new allergy checking strategy can be added. If payment integration changes, the payment adapter can be replaced.

Third, it supports testability. Service and domain classes can be tested without running the full console application. Repository and gateway interfaces can be replaced with fake or test implementations.

Finally, it supports extensibility. The system can later be extended with a web interface, database repositories, advanced allergy rules, real payment integration, or additional notification mechanisms without completely redesigning the application.

4.9 Implementation Overview and Code Structure

The system is implemented as a Java console application. The implementation follows the layered architecture explained in the previous sections. The code is organized into four main packages: presentation, application, domain, and infrastructure. This package structure makes the architectural separation visible in the source code.

The presentation package contains the console-based user interface classes. ConsoleApp controls the main application flow, CustomerMenu handles customer operations, AdminMenu handles admin operations, InputReader reads and validates

user input, and ConsolePrinter displays products, carts, orders, customers, and operation results.

The application package contains service and facade classes. CustomerService manages registration and login. CatalogService provides product listing operations. CartService manages cart operations and allergy-aware product addition. AllergyCheckService coordinates allergy checking through the strategy interface. PaymentService validates and processes payment requests. OrderService creates orders and manages order status transitions. AdminService manages product, customer, and order administration. CheckoutFacade coordinates the complete checkout workflow.

The domain package contains the core business model of the system. Classes such as Customer, AllergyProfile, Product, Cart, CartItem, Order, OrderItem, Money, PaymentRequest, PaymentResult, CheckoutResult, and OperationResult represent the main concepts of the system. The domain layer also contains repository interfaces, payment gateway abstraction, allergy checking strategy, order state classes, observers, builders, and factories.

The infrastructure package contains technical implementations. In-memory repository classes store customers, products, carts, and orders during runtime. PaymentGatewayAdapter connects the internal payment model with the simulated external payment gateway. ApplicationContext creates and connects the main dependencies of the system manually. ProductSeeder initializes the application with sample products.

This implementation structure shows that the Java code is not written as one large procedural console program. Instead, user interaction, business logic, domain rules, and infrastructure details are separated into different classes and packages.

4.10 Key Classes and Component Interaction

The main components of the system interact according to the layered architecture. Presentation classes call application services, application services coordinate domain objects and repository interfaces, and infrastructure classes provide concrete implementations for data storage and payment integration.

Customer-related operations start in CustomerMenu and are handled by CustomerService. During registration, CustomerService validates user information, creates an AllergyProfile, uses CustomerBuilder to create a Customer object, saves the customer through CustomerRepository, and creates an empty Cart through CartRepository.

Cart-related operations are coordinated by CartService. When a customer tries to add a product to the cart, CartService checks product availability, stock quantity, and customer allergy information. AllergyCheckService uses AllergyCheckStrategy to compare product allergens with the customer allergy profile. If a warning is detected, the item is added only when the customer confirms the risk.

Checkout is handled through CheckoutFacade. This facade coordinates CartService, OrderService, and PaymentService. It validates the cart, processes payment, creates the order, updates the order status, decreases stock, and clears the cart after successful checkout.

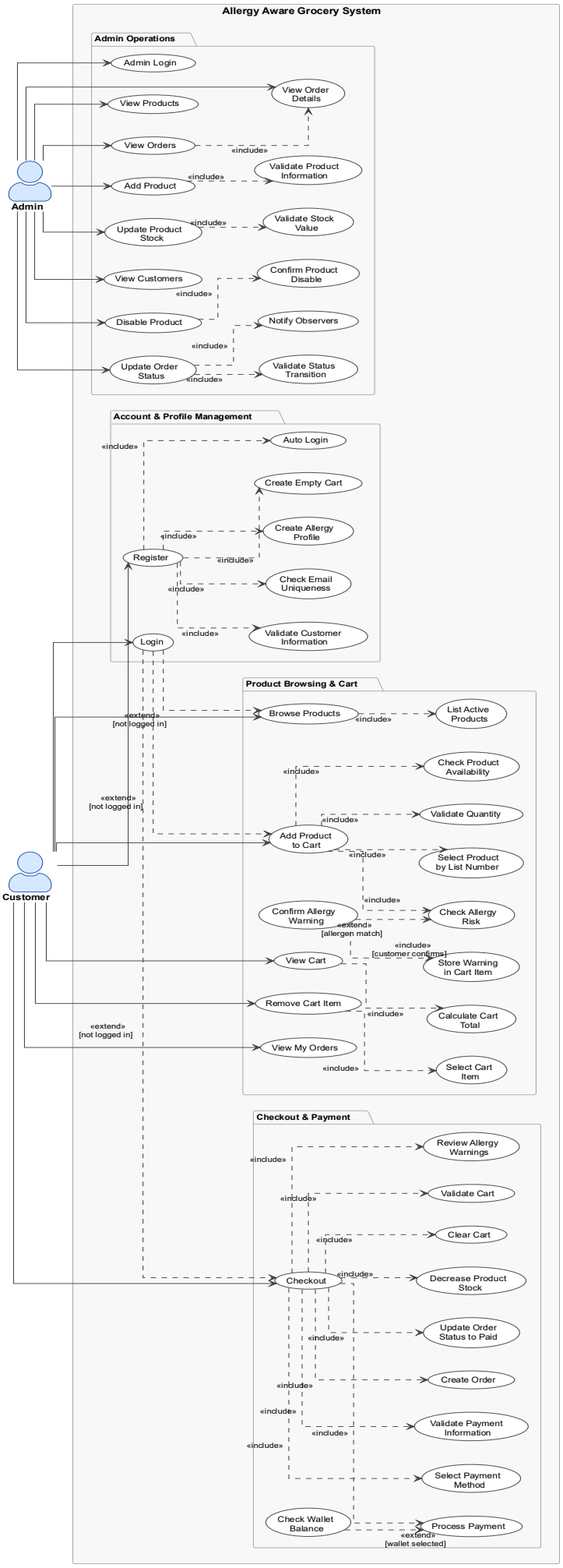
Order status management is handled by Order and OrderState classes. Concrete state classes such as CreatedState, PaidState, PreparingState, ShippedState, DeliveredState, and CancelledState control valid transitions. When an order status changes successfully, OrderObserver implementations such as CustomerNotificationObserver and OrderLogObserver are notified.

Admin operations are handled by AdminMenu and AdminService. AdminService uses ProductFactory for product creation, product repositories for stock and product status updates, and OrderService for order lifecycle changes.

5.UML Diagrams

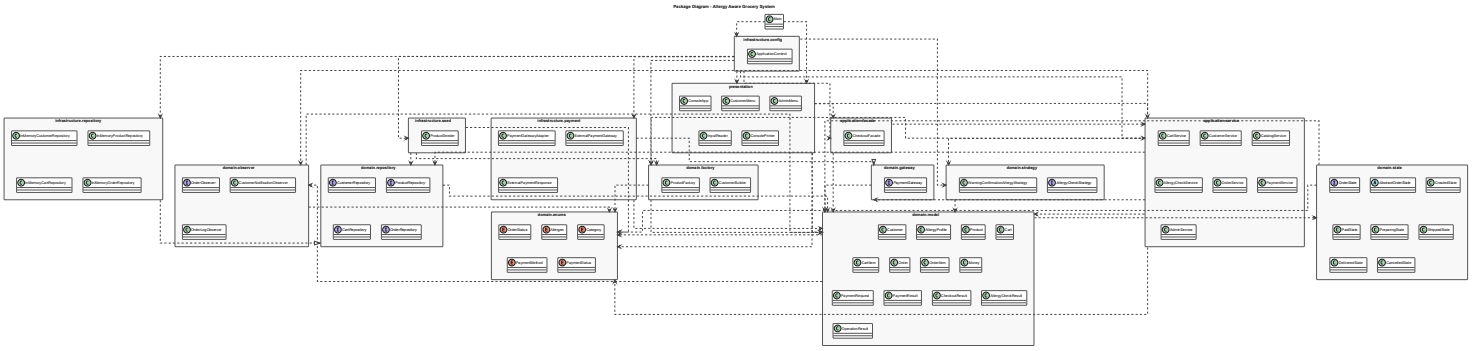
5.1 Use Case Diagram

The use case diagram shows the main interactions between the system actors and the Allergy Aware Grocery System. The two main actors are Customer and Admin. Customer-related use cases include registration, login, product browsing, allergy-aware cart operations, checkout, payment, and viewing orders. Admin-related use cases include product management, customer viewing, order viewing, and order status updates.


Allergy Aware Grocery System
Admin Operations
Admin Login
View Order Details
View Products
View Orders
«include»
Validate Product Information
Add Product
«include»
Admin
Update Product Stock
«include»
Validate Stock Value
View Customers
Confirm Product Disable
«include»
Disable Product
Notify Observers
«include»
Update Order Status
«include»
Validate Status Transition
Account & Profile Management
«include»
Auto Login
Create Empty Cart
Create Allergy Profile
«include»
Register
«include»
Check Email Uniqueness
«include»
Validate Customer Information
«include»
Login
Product Browsing & Cart
Browse Products
«include»
List Active Products
«extend»
[not logged in]
Check Product Availability
«include»
«extend»
[not logged in]
Validate Quantity
«include»
Add Product to Cart
«include»
Select Product by List Number
Customer
«include»
Confirm Allergy Warning
Check Allergy Risk
«extend»
[allergen match]
«include»
[customer confirms]
View Cart
Store Warning in Cart Item
Remove Cart Item
«include»
Calculate Cart Total
«extend»
[not logged in]
View My Orders
«include»
Select Cart Item
Checkout & Payment
Review Allergy Warnings
«include»
Validate Cart
«include»
Clear Cart
«include»
Checkout
«include»
Decrease Product Stock
Update Order Status to Paid
«include»
Create Order
«include»
«include»
Validate Payment Information
«include»
«include»
Select Payment Method
Check Wallet Balance
Process Payment
«extend»
[wallet selected]

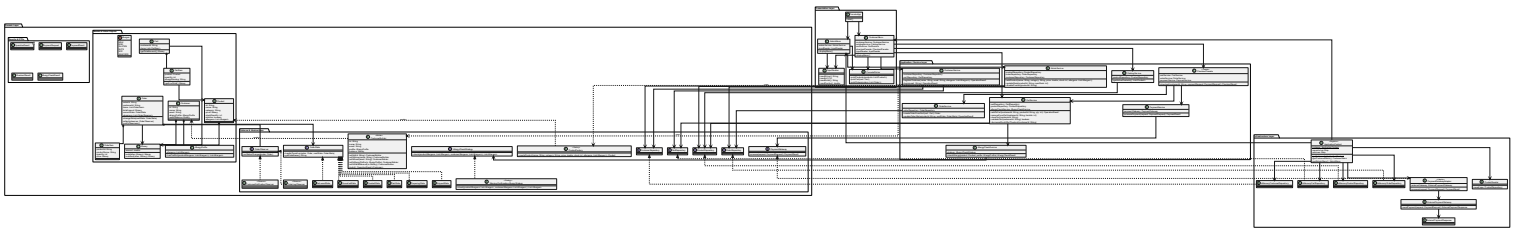
5.2 Package Diagram

The package diagram shows the layered structure of the system. The project is organized according to a layered architecture with presentation, application/service, domain, and infrastructure layers. This separation helps to keep user interaction, business logic, domain rules, and technical implementations independent from each other. You can see this diagram's bigger version in the appendix A.



5.3 Class Diagram

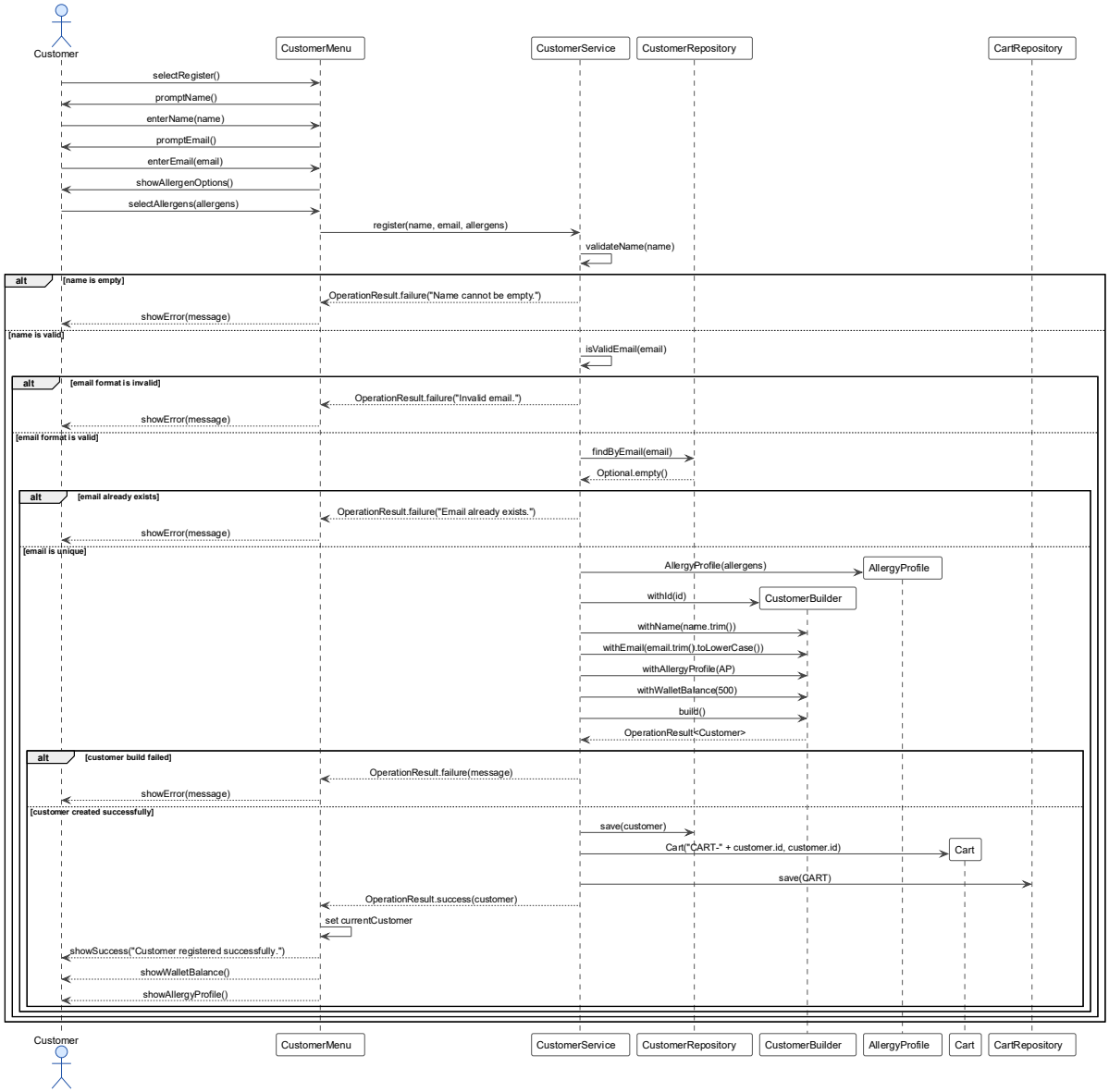
The class diagram presents the main classes, interfaces, enumerations, and relationships of the system. It includes domain model classes, service classes, repository abstractions, in-memory repository implementations, and the classes related to the applied design patterns. You can see this diagram's bigger version in the appendix B.



5.4 Sequence Diagrams

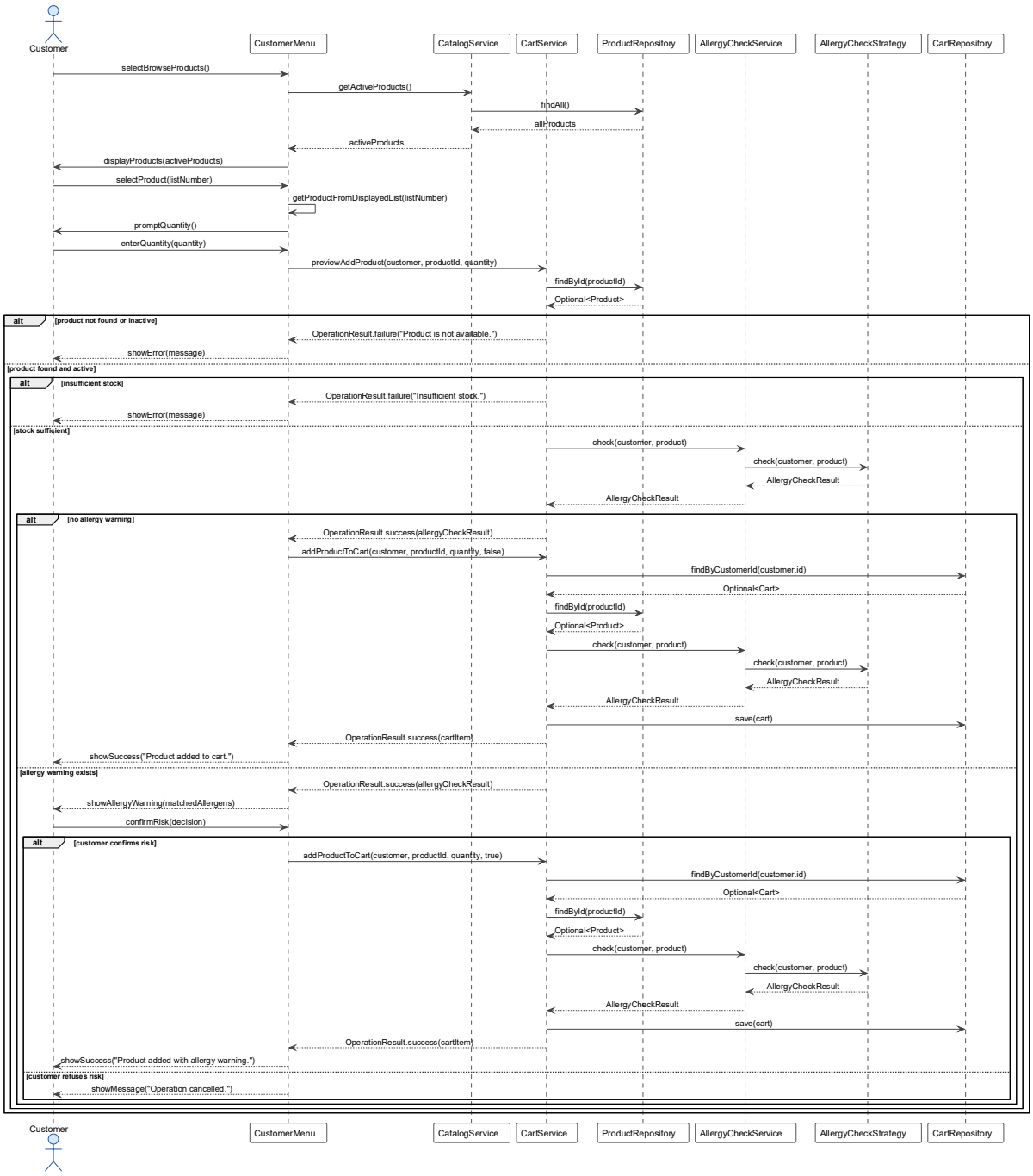
5.4.1 Sequence Diagram – Register and Create Allergy Profile

This sequence diagram shows how a new customer registers to the system. The customer enters name, email, and allergy information through CustomerMenu. CustomerService validates the information, checks email uniqueness through CustomerRepository, creates an AllergyProfile, builds the Customer object using CustomerBuilder, saves the customer, and creates an empty cart for the customer.



5.4.2 Sequence Diagram – Browse Products and Add Product to Cart

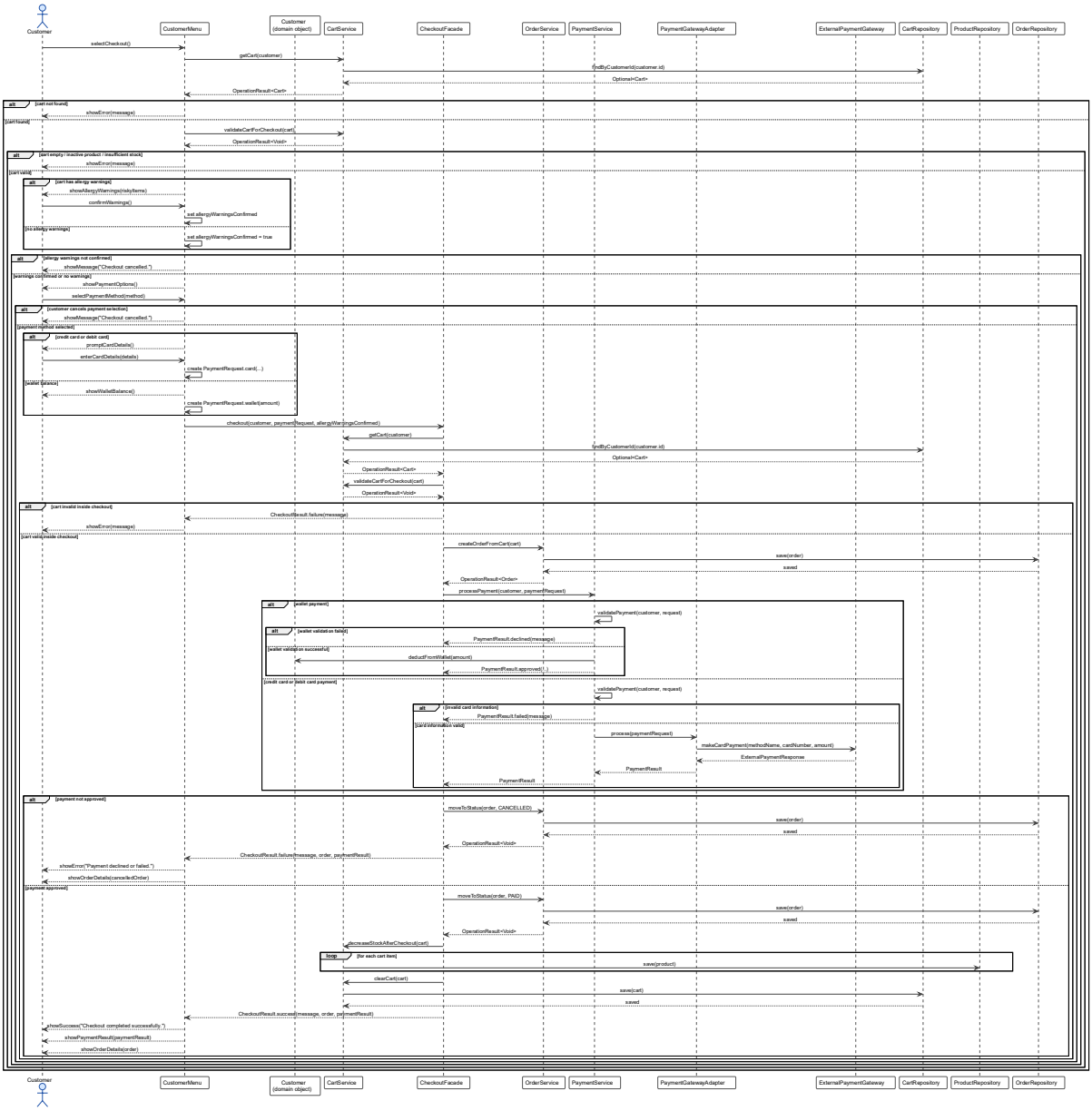
This sequence diagram shows the allergy-aware add-to-cart process. The customer selects a product from the displayed active product list and enters a quantity. CartService validates product availability and stock. Then AllergyCheckService uses the allergy check strategy to compare the product allergens with the customer allergy profile. If a risk is detected, the customer must explicitly confirm the warning before the item is added to the cart.



5.4.3 Sequence Diagram – Checkout and Payment Processing

This sequence diagram shows the checkout process coordinated by CheckoutFacade. The cart is validated before checkout, allergy warnings are confirmed if necessary, and the customer selects a payment method. PaymentService handles wallet and card payments. For card payments, PaymentGatewayAdapter communicates with the external payment gateway. If payment is approved, the order is moved to PAID status, product stock is decreased, and the cart is cleared. If payment fails, the order is

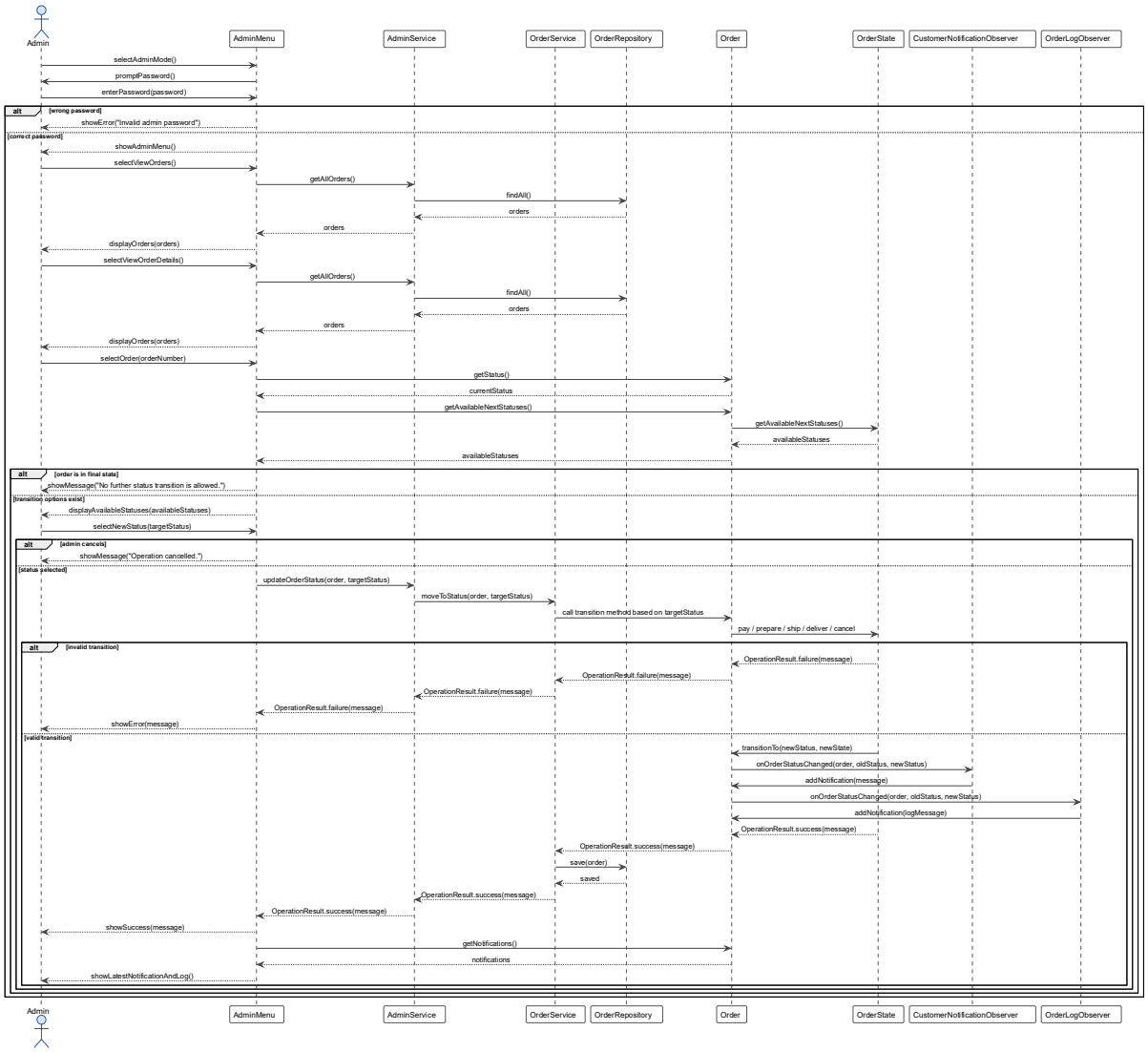
cancelled.



5.4.4 Sequence Diagram – Admin Updates Order Status

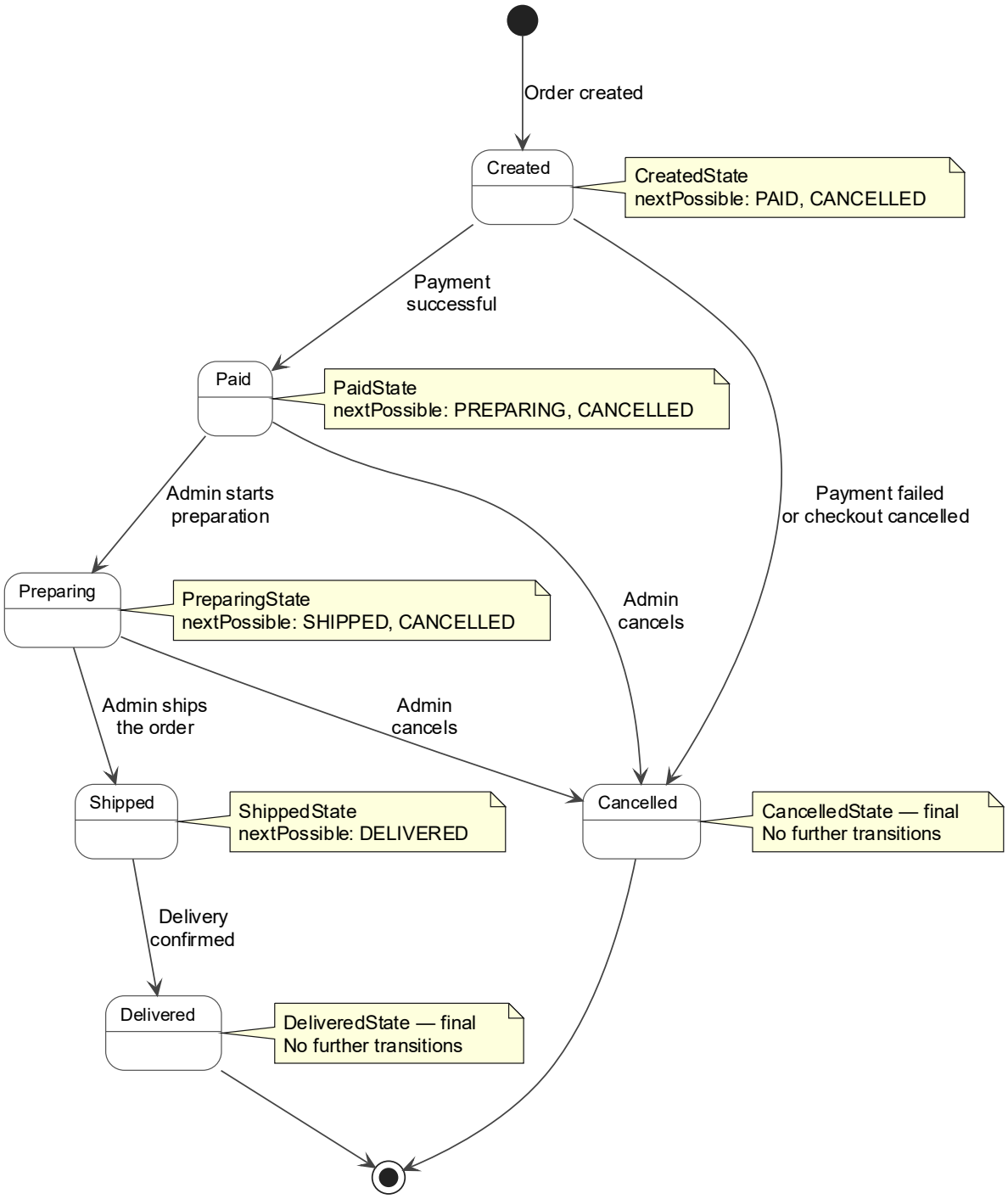
This sequence diagram shows how an admin updates an order status. After admin authentication, the admin views orders and selects an order. The system displays available next statuses according to the current order state. If the selected transition is valid, OrderService updates the order through the State pattern and the observers are notified. If the transition is invalid, the system returns an error and no notification is

created.



5.5 Statechart Diagram

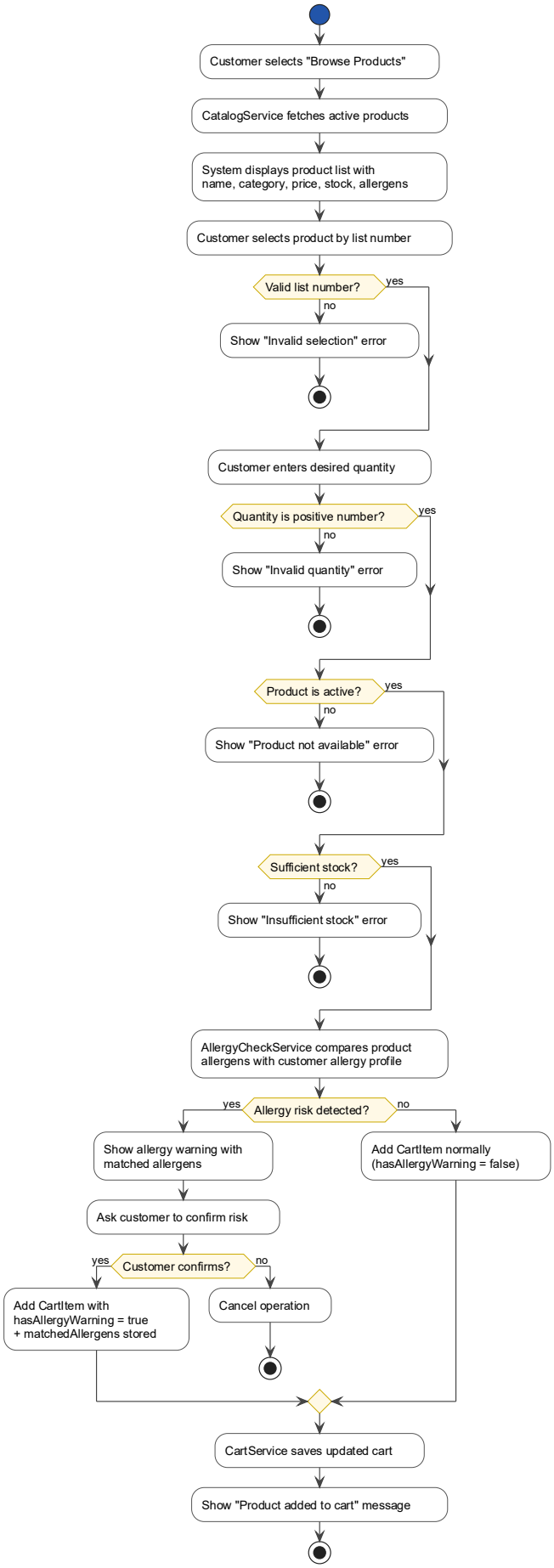
The statechart diagram shows the lifecycle of an order. Each order starts in the CREATED state. After successful payment, it moves to PAID. Then the admin can move it to PREPARING, SHIPPED, and DELIVERED. The order can also move to CANCELLED from allowed states such as CREATED, PAID, or PREPARING. DELIVERED and CANCELLED are final states.



5.6 Activity Diagrams

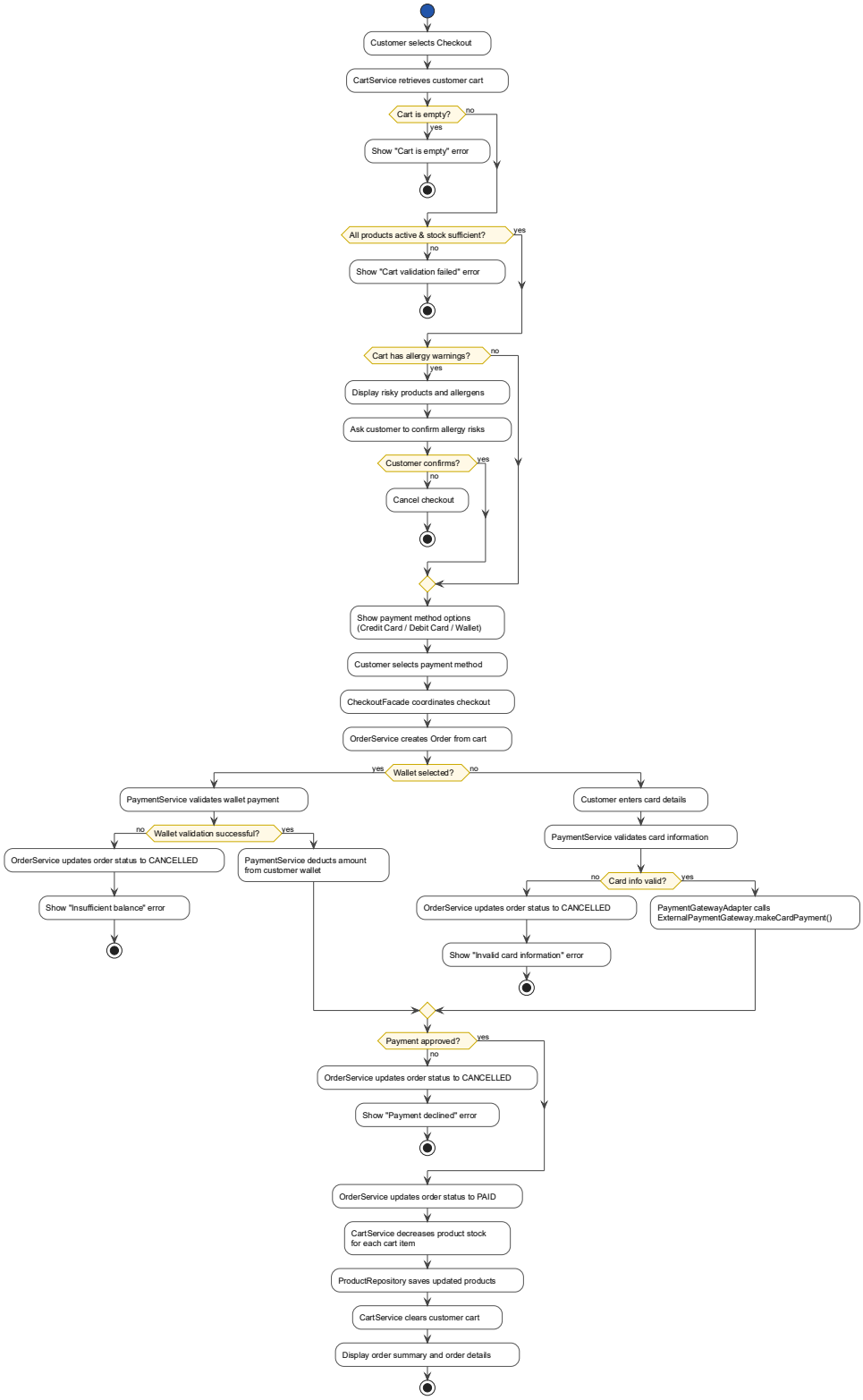
5.6.1 Activity Diagram – Allergy-Aware Add to Cart

This activity diagram represents the product selection and allergy checking workflow. It shows how the system validates the selected product, quantity, product availability, stock level, and allergy risk before adding the item to the cart.


Customer selects "Browse Products"
CatalogService fetches active products
System displays product list with name, category, price, stock, allergens
Customer selects product by list number
Valid list number?
yes
no
Show "Invalid selection" error
Customer enters desired quantity
Quantity is positive number?
yes
no
Show "Invalid quantity" error
Product is active?
yes
no
Show "Product not available" error
Sufficient stock?
yes
no
Show "Insufficient stock" error
AllergyCheckService compares product allergens with customer allergy profile
Allergy risk detected?
yes
no
Show allergy warning with matched allergens
Add CartItem normally (hasAllergyWarning = false)
Ask customer to confirm risk
Customer confirms?
yes
no
Add CartItem with hasAllergyWarning = true + matchedAllergens stored
Cancel operation
CartService saves updated cart
Show "Product added to cart" message

5.6.2 Activity Diagram – Checkout

This activity diagram represents the checkout workflow. It includes cart validation, allergy warning confirmation, payment method selection, payment validation, order status update, stock decrease, and cart clearing.



6. Design Patterns Used

6.1 Overview of Applied Design Patterns

This project uses several design patterns to improve maintainability, extensibility, and separation of responsibilities. Since the Allergy Aware Grocery System includes customer registration, product creation, allergy checking, checkout, payment processing, and order status management, different parts of the system have different design needs. For this reason, the project does not rely on only one pattern. Instead, it applies creational, structural, and behavioral design patterns together.

The creational patterns used in the project are Singleton, Builder, and Factory Method. Singleton is used in ApplicationContext to create and manage the main object graph of the application. Builder is used in CustomerBuilder to construct customer objects in a controlled way. Factory Method is used in ProductFactory to centralize product creation.

The structural patterns used in the project are Facade and Adapter. Facade is used in CheckoutFacade to simplify the checkout process for the presentation layer. Adapter is used in PaymentGatewayAdapter to connect the system's internal payment model with the simulated external payment gateway.

The behavioral patterns used in the project are Strategy, Observer, and State. Strategy is used for allergy checking. Observer is used to react to order status changes by creating customer notifications and order logs. State is used to control valid order status transitions such as CREATED, PAID, PREPARING, SHIPPED, DELIVERED, and CANCELLED.

The design pattern participants are represented in the class diagram, sequence diagrams, and statechart diagram in the UML section. The class diagram shows the structural relationships between pattern classes, while the sequence diagrams show how these patterns work during runtime.

6.2 Singleton Pattern – ApplicationContext

Category: Creational Pattern

Used in: ApplicationContext

The Singleton pattern is used in the ApplicationContext class. This class acts as the central configuration and object creation point of the application. It creates repository implementations, services, the payment gateway adapter, the product factory, and the

checkout facade. It also connects these objects manually through constructor-based dependency injection.

The main reason for selecting Singleton in this project is that the console application needs one shared application context during runtime. The system uses in-memory repositories, so it is important that the same repository instances are shared by the services. For example, if CustomerService saves a customer into a customer repository, AdminService should see the same customer data later. If each service created its own repository instance, the application data would become inconsistent.

ApplicationContext prevents this problem by creating one common object graph. It also makes the Main class cleaner because Main does not need to manually create every repository and service. Instead, it gets the required objects from ApplicationContext and starts the console application.

In the UML class diagram, ApplicationContext is shown as a singleton class and it has dependencies to repositories, services, CheckoutFacade, PaymentGatewayAdapter, ExternalPaymentGateway, and ProductSeeder.

The trade-off of this pattern is that Singleton introduces a global access point. If it is overused, it can make testing harder because classes may become tightly connected to the singleton instance. In this project, this risk is limited because the singleton is mainly used as a simple composition root. Business logic is still kept inside service and domain classes, not inside ApplicationContext.

6.3 Builder Pattern – CustomerBuilder

Category: Creational Pattern

Used in: CustomerBuilder

The Builder pattern is used to create Customer objects. A customer object contains several pieces of information, such as id, name, email, allergy profile, and wallet balance. Creating a customer directly with a long constructor call would make the code harder to read and easier to misuse.

CustomerBuilder solves this problem by providing step-by-step methods such as withId(), withName(), withEmail(), withAllergyProfile(), and withWalletBalance(). After the required values are set, the build() method creates the customer object and returns an OperationResult<Customer>. This makes customer creation more controlled and easier to understand.

The Builder pattern is especially useful in the registration use case. During registration, CustomerService validates the customer information, creates an AllergyProfile, and then uses CustomerBuilder to create the final Customer object. This keeps object construction separate from service flow logic.

In the UML class diagram, CustomerBuilder is connected to Customer, AllergyProfile, and OperationResult. In the registration sequence diagram, the builder appears when CustomerService creates a new customer after validation and email uniqueness checks.

The trade-off is that Builder adds an extra class to the system. For a very small class, this may be unnecessary. However, in this project, customer creation includes validation result handling, allergy profile assignment, and wallet initialization, so using a builder makes the creation process cleaner.

6.4 Factory Method Pattern – ProductFactory

Category: Creational Pattern

Used in: ProductFactory

The Factory Method pattern is used in ProductFactory to create Product objects. Product creation requires several values, such as name, category, price, stock quantity, and allergen information. It also involves product id generation and product validation. Instead of spreading this logic across the admin menu or admin service, product creation is centralized in ProductFactory.

The main reason for selecting this pattern is to separate product creation logic from product management logic. AdminService is responsible for admin operations such as adding a product, updating stock, disabling products, and viewing products. It should not contain all the details of how a valid Product object is created. ProductFactory handles this responsibility.

This design also makes the system easier to extend. If product creation rules change later, for example if different categories need different default values, this change can be handled mainly inside the factory. The rest of the application can continue to request products from the factory.

In the UML class diagram, ProductFactory is connected to Product, Category, Allergen, Money, and OperationResult. It is also used by AdminService and ProductSeeder.

The trade-off of this pattern is that the factory can become too large if too many creation rules are added to it. If the system grows and product categories become more

complex, the factory may need to be divided into smaller specialized factories. For the current project scope, a single ProductFactory is sufficient.

6.5 Strategy Pattern – AllergyCheckStrategy

Category: Behavioral Pattern

Used in: AllergyCheckStrategy, WarningConfirmationAllergyStrategy, AllergyCheckService

The Strategy pattern is used for allergy checking. The system must compare a product's allergens with the customer's allergy profile and decide whether a warning should be shown. This rule is important because the main purpose of the system is to make grocery shopping safer for allergy-sensitive customers.

The strategy interface is AllergyCheckStrategy. The concrete strategy is WarningConfirmationAllergyStrategy. AllergyCheckService depends on the strategy interface instead of directly depending on one fixed allergy checking implementation. This means the allergy checking behavior can be changed without rewriting the service class.

In the current version, the strategy checks whether the product allergens match any allergens in the customer's allergy profile. If there is a match, it returns an AllergyCheckResult containing warning information and matched allergens. Then the customer must confirm the risk before the product can be added to the cart.

This pattern was selected because allergy checking may evolve in future versions. For example, the system could later support severity levels, cross-contamination warnings, medical profile rules, or alternative product suggestions. With the Strategy pattern, a new allergy checking algorithm can be added as another strategy class.

In the UML class diagram, AllergyCheckStrategy is shown as an interface, and WarningConfirmationAllergyStrategy implements it. In the add-to-cart sequence diagram, CartService uses AllergyCheckService, and AllergyCheckService delegates the actual checking operation to the strategy.

The trade-off is that the current system has only one concrete strategy. Therefore, the pattern may look slightly more complex than necessary at first. However, because allergy checking is a central and changeable business rule, using Strategy is reasonable for maintainability and extensibility.

6.6 Facade Pattern – CheckoutFacade

Category: Structural Pattern

Used in: CheckoutFacade

The Facade pattern is used in CheckoutFacade. Checkout is one of the most complex flows in the system because it involves several steps: retrieving the cart, validating the cart, checking allergy warnings, creating an order, processing payment, updating order status, decreasing stock, and clearing the cart.

Without a facade, CustomerMenu would have to call CartService, OrderService, PaymentService, and repository operations separately. This would make the presentation layer too complex and would mix user interaction logic with checkout business logic.

CheckoutFacade provides a simplified method:

checkout(customer, paymentRequest, allergyWarningsConfirmed)

The presentation layer calls this method and receives a CheckoutResult. The facade coordinates the required services internally. This keeps CustomerMenu cleaner and makes the checkout process easier to understand.

In the UML class diagram, CheckoutFacade depends on CartService, OrderService, and PaymentService. In the checkout sequence diagram, the facade coordinates the checkout flow after the customer selects a payment method.

The trade-off is that a facade can become too large if too much business logic is placed inside it. To avoid this, the project keeps detailed responsibilities inside the related services. For example, cart validation remains in CartService, order creation remains in OrderService, and payment processing remains in PaymentService. The facade mainly coordinates these services.

6.7 Adapter Pattern – PaymentGatewayAdapter

Category: Structural Pattern

Used in: PaymentGateway, PaymentGatewayAdapter, ExternalPaymentGateway

The Adapter pattern is used to integrate the system with a simulated external payment gateway. The internal application works with the PaymentGateway interface and expects a PaymentResult. However, the external gateway has a different method and response structure. It uses makeCardPayment() and returns an ExternalPaymentResponse.

PaymentGatewayAdapter adapts the external gateway to the internal payment interface. PaymentService does not need to know the details of the external gateway. It only calls the PaymentGateway interface. The adapter translates the system's PaymentRequest into the format expected by ExternalPaymentGateway, calls the external method, and converts the external response into a PaymentResult.

This pattern was selected because payment systems often come from external providers, and their APIs are usually different from the application's internal model. By using an adapter, the system protects its service layer from external API details.

In the UML class diagram, PaymentGatewayAdapter implements PaymentGateway and depends on ExternalPaymentGateway and ExternalPaymentResponse. In the checkout sequence diagram, the adapter is used only for credit card and debit card payments. Wallet payment is handled directly by PaymentService.

The trade-off is that the adapter adds another class and mapping responsibility. If the external API changes, the adapter must be updated. However, this is still better than spreading external gateway details across the payment service or checkout logic.

6.8 Observer Pattern – OrderObserver

Category: Behavioral Pattern

Used in: OrderObserver, CustomerNotificationObserver, OrderLogObserver, Order

The Observer pattern is used to react to order status changes. When an order status changes, the system needs to create customer notifications and order log messages. These actions should not be hard-coded directly into every status transition method.

The observer interface is OrderObserver. The concrete observers are CustomerNotificationObserver and OrderLogObserver. The Order object keeps a list of observers and notifies them when its status changes successfully. The observers then add notification or log messages to the order.

This pattern was selected because status change side effects may increase in the future. For example, the system could later send email notifications, SMS messages, admin dashboard alerts, or audit records. With the Observer pattern, these new reactions can be added as new observer classes without changing the main order transition logic too much.

In the UML class diagram, Order has a relationship with OrderObserver, and both concrete observer classes implement the observer interface. In the admin order status

sequence diagram, observers are triggered only after a valid status transition. If the transition is invalid, observers are not notified.

The trade-off is that observer behavior can make the flow less direct because some actions happen indirectly after a state change. In this project, the observer mechanism is kept simple and synchronous, so it remains understandable.

6.9 State Pattern – OrderState

Category: Behavioral Pattern

Used in: OrderState, AbstractOrderState, CreatedState, PaidState, PreparingState, ShippedState, DeliveredState, CancelledState, Order

The State pattern is used to manage the order lifecycle. An order can move through several states: CREATED, PAID, PREPARING, SHIPPED, DELIVERED, and CANCELLED. Not every transition is valid. For example, an order can move from PAID to PREPARING, but it should not move directly from CREATED to SHIPPED.

Instead of placing all transition rules inside a large conditional block, the project represents each order status with a separate state class. The OrderState interface defines operations such as pay(), prepare(), ship(), deliver(), and cancel(). Concrete state classes decide which operations are valid.

For example, CreatedState allows the order to move to PAID or CANCELLED. PaidState allows the order to move to PREPARING or CANCELLED. PreparingState allows the order to move to SHIPPED or CANCELLED. ShippedState allows the order to move to DELIVERED. DeliveredState and CancelledState are final states.

This pattern was selected because order status management is a clear state-based problem. It makes invalid transitions easier to prevent and keeps transition logic close to the related state class.

In the UML class diagram, OrderState is shown as an interface, AbstractOrderState implements common behavior, and concrete state classes extend it. The statechart diagram also represents the valid order lifecycle visually.

The trade-off is that the State pattern creates several small classes. For a very simple system, a single enum with if-else statements might be shorter. However, the State pattern gives a cleaner structure when valid transitions and state-specific behavior become more important.

6.10 Summary of Pattern Usage

The project uses more than the minimum required number of design patterns. These patterns are not added only for demonstration; each one supports a real design need in the system.

Singleton helps the application share one object graph and one set of in-memory repositories. Builder and Factory Method make object creation more controlled. Strategy separates allergy checking rules from the service layer. Facade simplifies the checkout process for the presentation layer. Adapter protects the application from external payment gateway details. Observer handles side effects of order status changes. State manages the order lifecycle and prevents invalid transitions.

Together, these patterns improve modularity, readability, maintainability, and extensibility. They also support the layered architecture because presentation classes do not directly handle domain rules, repository details, payment integration, or order transition logic.

7. SOLID Evaluation

7.1 Single Responsibility Principle

The Single Responsibility Principle states that a class should have only one main reason to change. In other words, each class should focus on a clear responsibility instead of handling many unrelated tasks.

The project generally follows this principle by separating the system into different layers and assigning different responsibilities to different classes. For example, CustomerService is responsible for customer registration and login operations. CartService is responsible for cart operations such as adding products, removing items, validating the cart, decreasing stock after checkout, and clearing the cart. PaymentService is responsible for payment validation and payment processing. OrderService is responsible for order creation and order status updates. This separation prevents the whole system from being implemented inside one large class.

A good example is the checkout process. The checkout operation is complex, but the responsibilities are distributed. CheckoutFacade coordinates the process, CartService validates the cart, OrderService creates and updates the order, and PaymentService handles payment logic.

CheckoutFacade
- coordinates checkout flow

CartService
- validates cart
- decreases stock
- clears cart

OrderService
- creates order
- changes order status

PaymentService
- validates and processes payment

This structure makes the code easier to understand and modify. For example, if payment rules change, the main class to update is PaymentService, not CustomerMenu, CartService, or OrderService.

A potential weakness is that some menu classes, especially CustomerMenu and AdminMenu, contain long user interaction flows. They handle input reading, validation messages, menu navigation, and service calls in the same class. This is acceptable for a console-based school project, but in a larger system it would make the presentation layer harder to maintain.

An improvement would be to divide menu responsibilities into smaller helper classes. For example, input validation, output formatting, and menu navigation could be separated into different classes. Also, admin authentication could be moved from AdminMenu into a separate AdminAuthService.

7.2 Open-Closed Principle

The Open-Closed Principle states that software entities should be open for extension but closed for modification. This means that new behavior should be added by extending the system rather than changing existing working code too much.

The project follows this principle in several places by using design patterns. The AllergyCheckStrategy interface makes allergy checking extensible. The current concrete strategy is WarningConfirmationAllergyStrategy, but another strategy could be added later without rewriting AllergyCheckService.

```
public interface AllergyCheckStrategy {

   AllergyCheckResult check(Customer customer, Product product);

}
```

For example, the system could later add:

SeverityBasedAllergyStrategy
CrossContaminationAllergyStrategy
AlternativeSuggestionAllergyStrategy

These new classes could implement AllergyCheckStrategy, and AllergyCheckService could continue to depend on the interface.

The State pattern also supports the Open-Closed Principle. Order transition rules are separated into state classes such as CreatedState, PaidState, PreparingState, ShippedState, DeliveredState, and CancelledState. Each state controls its own valid transitions. This is cleaner than writing all transition rules in one large if-else block inside the Order class.

The Observer pattern also supports this principle. The system currently has CustomerNotificationObserver and OrderLogObserver. If a new notification behavior is needed later, such as email notification, it can be added as a new observer class without changing the main order transition logic too much.

A potential violation exists in OrderService.moveToStatus(). Since this method uses a selection structure based on the target order status, adding a new order status may require modifying this method. This means the method is not completely closed for modification.

A possible improvement would be to reduce the switch or conditional logic by moving more transition responsibility into the state objects or by using a command-based transition handler. However, for the current project size, the existing implementation is still understandable and acceptable.

7.3 Liskov Substitution Principle

The Liskov Substitution Principle states that objects of a superclass or interface should be replaceable with objects of its subclasses or implementations without breaking the correctness of the program.

The project follows this principle mainly through interfaces and their concrete implementations. For example, service classes depend on repository interfaces such as CustomerRepository, ProductRepository, CartRepository, and OrderRepository. The concrete in-memory repository implementations can be used through these interfaces.

CustomerRepository customerRepository = new InMemoryCustomerRepository();

ProductRepository productRepository = new InMemoryProductRepository();

CartRepository cartRepository = new InMemoryCartRepository();

OrderRepository orderRepository = new InMemoryOrderRepository();

Since the services use repository interfaces, another implementation could replace the in-memory repository later. For example, a database-based repository could be added without changing the service logic, as long as it follows the same interface contract.

The State pattern is another good example. CreatedState, PaidState, PreparingState, ShippedState, DeliveredState, and CancelledState can all be used wherever an OrderState is expected. The Order object does not need to know the exact concrete state class in order to call methods such as pay(), prepare(), ship(), deliver(), or cancel().

private OrderState state;

This means the Order class can delegate behavior to the current state object without depending on one specific state class.

A potential risk would appear if one concrete state class did not respect the expected behavior of OrderState. For example, if an invalid transition caused an unexpected exception instead of returning an OperationResult.failure(), then substituting that state object could break the system flow.

The improvement is to keep the contract of OrderState clear: each state method should either perform a valid transition and return success, or reject an invalid transition by returning a failure result. All concrete state classes should follow this behavior consistently.

7.4 Interface Segregation Principle

The Interface Segregation Principle states that a class should not be forced to depend on methods it does not use. Instead of creating large general-purpose interfaces, the system should use smaller and more specific interfaces.

The project follows this principle by separating repository interfaces according to entity type. Instead of using one large repository interface for all data operations, the project has separate interfaces:

CustomerRepository
ProductRepository

CartRepository
OrderRepository

Each repository interface contains operations related to its own domain object. For example, CustomerRepository contains customer-related methods such as saving a customer and finding a customer by email. ProductRepository contains product-related methods. OrderRepository contains order-related methods.

This is better than having one large interface like:

Repository
- saveCustomer()
- findCustomerByEmail()
- saveProduct()
- findProductById()
- saveCart()
- findCartByCustomerId()
- saveOrder()
- findOrdersByCustomerId()

Such a large interface would force classes to depend on unrelated methods. The current design avoids this problem.

The payment abstraction also follows this principle. PaymentGateway only contains the payment processing behavior needed by PaymentService. It does not include unrelated methods such as customer management, reporting, or order operations.

A potential weakness is that some service classes still contain several public methods because they represent complete use-case areas. For example, CartService handles adding products, removing items, validating carts, decreasing stock, and clearing carts. These operations are all related to cart management, so this is not a serious violation. However, if the class grows more in future versions, it may become too broad.

An improvement would be to split very large service interfaces or service responsibilities if the system expands. For example, checkout-related cart validation and inventory stock update behavior could be separated into specialized services such as CartValidationService or InventoryService.

7.5 Dependency Inversion Principle

The Dependency Inversion Principle states that high-level modules should not depend directly on low-level concrete classes. Both should depend on abstractions. It also

states that abstractions should not depend on details; details should depend on abstractions.

The project follows this principle by making service classes depend on interfaces instead of concrete infrastructure classes. For example, CustomerService depends on CustomerRepository and CartRepository, not directly on InMemoryCustomerRepository or InMemoryCartRepository.

```
public class CustomerService {

   private final CustomerRepository customerRepository;

   private final CartRepository cartRepository;

}
```

This is important because the service layer should not know how data is stored. In the current version, data is stored in memory. However, if the project later used a database, the service layer would not need major changes. Only the repository implementation would change.

PaymentService also follows this principle because it depends on the PaymentGateway interface. The concrete adapter class, PaymentGatewayAdapter, implements this interface and connects the internal system to ExternalPaymentGateway.

```
public class PaymentService {

   private final PaymentGateway paymentGateway;

}
```

This design prevents PaymentService from depending directly on the external payment gateway. The external gateway is an infrastructure detail, and the adapter hides this detail from the application layer.

AllergyCheckService also supports Dependency Inversion because it depends on AllergyCheckStrategy, not directly on one concrete allergy checking algorithm.

```
public class AllergyCheckService {

   private final AllergyCheckStrategy strategy;

}
```

A potential weakness is that ApplicationContext directly creates concrete objects. However, this is not a serious violation in this project because ApplicationContext acts as the composition root. Its responsibility is to assemble the application manually. The rest of the system still depends mostly on abstractions.

An improvement for a larger application would be to use a dependency injection framework or configuration-based object creation. However, for this project, manual dependency injection is appropriate because the project is a console-based Java application and the focus is on demonstrating layered architecture clearly.

7.6 General SOLID Summary

Overall, the Allergy Aware Grocery System follows SOLID principles reasonably well. The system is divided into layers and responsibilities are separated between menu classes, services, domain models, repositories, strategies, states, observers, factories, and adapters.

The strongest principles in the project are Dependency Inversion and Single Responsibility. Service classes mostly depend on interfaces, and most classes have clear responsibilities. Open-Closed Principle is supported by the use of Strategy, Observer, State, Adapter, and Factory Method patterns. Liskov Substitution Principle is supported by repository, strategy, payment gateway, observer, and state interfaces. Interface Segregation is supported by small and specific repository and gateway interfaces.

The main improvement areas are related to the presentation layer and some conditional logic in service classes. Menu classes could be divided into smaller classes, and some status transition logic could be made more extensible. However, these limitations do not break the overall design. For the current project scope, the system demonstrates a clean and understandable object-oriented design.

8. Testing Strategy

8.1 Overview

The Allergy Aware Grocery System was tested with a scenario-based testing approach. Since the project is a console-based Java application and does not include a graphical user interface or a real database, the main testing focus is on service behavior, domain rules, validation logic, and complete user flows.

The system can be tested manually through the console menus, and the service and domain classes are also suitable for unit testing with JUnit. The most important goal of testing is to verify that customer registration, allergy checking, cart operations, checkout, payment processing, order status transitions, and admin operations work correctly under both valid and invalid conditions.

The testing strategy is divided into the following areas:
- Customer registration and login testing
- Product browsing and allergy check testing
- Cart operation testing
- Checkout and payment testing
- Order lifecycle testing
- Admin operation testing
- Input validation testing
- Edge case testing

8.2 Customer Registration and Login Testing

Customer registration is tested by entering both valid and invalid customer information. The system should not allow empty names, invalid e-mail addresses, or duplicate e-mail addresses. A valid customer should be saved successfully, and an empty cart should be created for the customer after registration.

Example test scenarios:

Test Case 1:
Input: Empty customer name
Expected Result: Registration fails and an error message is displayed.

Test Case 2:
Input: Invalid e-mail address without "@"
Expected Result: Registration fails and an invalid e-mail message is displayed.

Test Case 3:
Input: E-mail address that already exists
Expected Result: Registration fails because duplicate e-mail addresses are not allowed.

Test Case 4:
Input: Valid name, valid e-mail, and selected allergens
Expected Result: Customer is created, allergy profile is assigned, and an empty cart is created.

Test Case 5:
Input: Existing registered e-mail during login
Expected Result: Login succeeds.

Test Case 6:
Input: Non-existing e-mail during login
Expected Result: Login fails and the system displays an error message.

These tests verify that CustomerService, CustomerRepository, AllergyProfile, CustomerBuilder, and CartRepository work together correctly during registration.

8.3 Product Browsing and Allergy Check Testing

The product browsing and allergy check process is one of the most important parts of the project. The system should display only active products to the customer and should warn the customer if the selected product contains allergens that match the customer's allergy profile.

Example test scenarios:

Test Case 1:
Input: Customer browses products
Expected Result: Only active products are displayed with product name, category, price, stock, and allergen information.

Test Case 2:
Input: Customer selects an invalid product list number
Expected Result: Product selection fails and an error message is displayed.

Test Case 3:
Input: Customer selects a valid product with no matching allergens
Expected Result: Product can be added to the cart without an allergy warning.

Test Case 4:
Input: Customer selects a product with allergens matching the customer allergy profile
Expected Result: Allergy warning is displayed with matched allergens.

Test Case 5:
Input: Customer confirms the allergy warning
Expected Result: Product is added to the cart with allergy warning information stored in the cart item.

Test Case 6:
Input: Customer rejects the allergy warning
Expected Result: Product is not added to the cart.

These tests verify the behavior of CatalogService, CartService, AllergyCheckService, AllergyCheckStrategy, and WarningConfirmationAllergyStrategy.

8.4 Cart Operation Testing

Cart operations are tested to make sure that products are added, removed, and calculated correctly. The system should also prevent invalid cart operations such as adding inactive products, adding a product with insufficient stock, or removing a non-existing cart item.

Example test scenarios:

Test Case 1:
Input: Add an active product with sufficient stock and valid quantity
Expected Result: Product is added to the cart.

Test Case 2:
Input: Add a product with quantity 0 or negative quantity
Expected Result: Product is not added and an invalid quantity message is displayed.

Test Case 3:
Input: Add a product with quantity greater than available stock
Expected Result: Product is not added and an insufficient stock message is displayed.

Test Case 4:
Input: Remove a valid cart item by item number
Expected Result: Selected item is removed from the cart.

Test Case 5:
Input: Remove an invalid cart item number
Expected Result: Remove operation fails and an error message is displayed.

Test Case 6:
Input: View cart after adding multiple products
Expected Result: Cart displays all items, quantities, line totals, allergy warning information, and total amount.

These tests verify Cart, CartItem, CartService, Product, and Money calculations.

8.5 Checkout and Payment Testing

Checkout testing verifies the complete order creation and payment flow. This is a critical flow because it affects order creation, payment result, product stock, and cart clearing.

The checkout process should validate the cart before payment. If the cart is empty, contains inactive products, has insufficient stock, or has unconfirmed allergy warnings, checkout should not continue. If payment fails, the created order should be moved to CANCELLED. If payment succeeds, the order should be moved to PAID, product stock should be decreased, and the cart should be cleared.

Example test scenarios:

Test Case 1:
Input: Checkout with an empty cart
Expected Result: Checkout fails and no order is completed.

Test Case 2:
Input: Checkout with a cart that contains insufficient stock
Expected Result: Checkout fails during cart validation.

Test Case 3:
Input: Checkout with allergy warnings not confirmed
Expected Result: Checkout is cancelled before payment.

Test Case 4:
Input: Wallet payment with insufficient wallet balance
Expected Result: Payment is declined and order is cancelled.

Test Case 5:
Input: Wallet payment with sufficient wallet balance
Expected Result: Wallet amount is deducted, order becomes PAID, stock decreases, and cart is cleared.

Test Case 6:
Input: Credit card payment with invalid card information
Expected Result: Payment fails and order is cancelled.

Test Case 7:
Input: Credit card or debit card payment approved by external gateway
Expected Result: Order becomes PAID, stock decreases, and cart is cleared.

Test Case 8:
Input: Card payment declined by external gateway
Expected Result: Order becomes CANCELLED and cart is not finalized as a successful checkout.

These tests verify CheckoutFacade, CartService, OrderService, PaymentService, PaymentGatewayAdapter, ExternalPaymentGateway, PaymentRequest, PaymentResult, and CheckoutResult.

8.6 Order Lifecycle and State Transition Testing

Order lifecycle testing verifies that the State pattern works correctly. The system should allow only valid transitions and block invalid transitions. Observer notifications should be created only after successful transitions.

Valid transition test scenarios:

Test Case 1:
Input: CREATED -> PAID
Expected Result: Transition succeeds.

Test Case 2:
Input: PAID -> PREPARING
Expected Result: Transition succeeds.

Test Case 3:
Input: PREPARING -> SHIPPED
Expected Result: Transition succeeds.

Test Case 4:
Input: SHIPPED -> DELIVERED
Expected Result: Transition succeeds.

Test Case 5:
Input: CREATED -> CANCELLED
Expected Result: Transition succeeds.

Test Case 6:
Input: PAID -> CANCELLED
Expected Result: Transition succeeds.

Test Case 7:
Input: PREPARING -> CANCELLED
Expected Result: Transition succeeds.

Invalid transition test scenarios:

Test Case 8:
Input: CREATED -> SHIPPED
Expected Result: Transition fails.

Test Case 9:
Input: DELIVERED -> CANCELLED
Expected Result: Transition fails because DELIVERED is a final state.

Test Case 10:
Input: CANCELLED -> PAID
Expected Result: Transition fails because CANCELLED is a final state.

Observer-related test scenario:

Test Case 11:
Input: Valid order status transition
Expected Result: CustomerNotificationObserver and OrderLogObserver add notification/log messages to the order.

Test Case 12:
Input: Invalid order status transition
Expected Result: No observer notification is created.

These tests verify Order, OrderState, concrete state classes, OrderService, CustomerNotificationObserver, and OrderLogObserver.

8.7 Admin Operation Testing

Admin testing verifies that administrative operations are protected and work correctly. The admin should be able to add products, update stock, disable products, view customers, view orders, and update order statuses.

Example test scenarios:

Test Case 1:
Input: Wrong admin password
Expected Result: Admin menu is not opened.

Test Case 2:
Input: Correct admin password
Expected Result: Admin menu is opened.

Test Case 3:
Input: Add product with valid name, category, price, stock, and allergens
Expected Result: Product is created and saved successfully.

Test Case 4:
Input: Add product with invalid price or invalid stock
Expected Result: Product creation fails.

Test Case 5:
Input: Update product stock with valid value
Expected Result: Product stock is updated.

Test Case 6:
Input: Update product stock with negative value
Expected Result: Stock update fails.

Test Case 7:
Input: Disable active product
Expected Result: Product becomes inactive and is no longer shown in active product browsing.

Test Case 8:
Input: Update order to one of its available next statuses
Expected Result: Order status is updated successfully.

Test Case 9:
Input: Try to update order to an invalid status
Expected Result: Transition fails and no observer notification is created.

These tests verify AdminMenu, AdminService, ProductFactory, ProductRepository, OrderService, and OrderRepository.

8.8 Input Validation and Edge Case Testing

The system includes several user input points, so invalid input testing is important. Since the project uses a console interface, the system should handle wrong menu choices and invalid numeric inputs without crashing.

Important input validation tests include:
- Invalid menu selection
- Non-numeric product list number
- Product list number outside the displayed range
- Non-numeric quantity input
- Zero or negative quantity
- Invalid e-mail format
- Duplicate e-mail address
- Invalid card number format
- Invalid stock value
- Invalid product price
- Invalid order selection number

Edge case testing also includes checking empty repositories, empty carts, final order states, and products with no allergens. These tests help verify that the system behaves safely in unusual situations, not only in the normal successful flow.

8.9 Testing Summary

The testing strategy focuses on validating the main business rules of the Allergy Aware Grocery System. The most important rules are allergy warning control, safe cart validation, successful and failed checkout behavior, payment handling, stock consistency, and valid order lifecycle transitions.

Manual scenario-based testing through the console is suitable for this academic project because the system does not use a GUI or a real database. However, the code structure also supports automated unit testing. Since services depend on repository and gateway abstractions, test implementations or mock objects could be used in JUnit tests. For example, PaymentGateway could be replaced with a fake gateway that returns approved or declined payment results, and repository interfaces could be replaced with in-memory test repositories.

Overall, the project is designed in a testable way because responsibilities are separated between services, domain objects, repositories, strategies, states, observers, and adapters. This separation makes it easier to test each part of the system individually and also test complete business scenarios from registration to checkout.

9. Limitations

9.1 Overview

Although the Allergy Aware Grocery System satisfies the main requirements of the term project, it still has some limitations. These limitations are mostly related to the project scope. The main focus of this project is object-oriented design, layered architecture, design patterns, SOLID principles, and UML modeling. Therefore, some real-world features such as a graphical user interface, database integration, advanced security, and real payment processing are intentionally kept outside the scope.

The current version works as a console-based Java application. It demonstrates the main business flows clearly, but it is not designed as a production-ready e-commerce system. The following subsections explain the main limitations of the system and possible future improvements.

9.2 Console-Based User Interface

The system uses a console-based user interface. Customers and admins interact with the system by entering menu numbers and text input. This is suitable for demonstrating the business logic and object-oriented structure of the project, but it is not as user-friendly as a graphical or web-based interface.

For example, product selection is done by entering the displayed list number. This keeps the implementation simple and avoids unnecessary complexity, but a real grocery system would normally use a web page or mobile application with product images, search filters, category pages, and buttons.

A future improvement would be to develop a web-based interface or a desktop GUI. The existing layered architecture would support this improvement because the presentation layer is separated from the service and domain layers. Therefore, the console menu could be replaced by another interface without rewriting the whole business logic.

9.3 In-Memory Data Storage

The project uses in-memory repositories instead of a real database. This means that customers, products, carts, and orders exist only while the application is running. When the program is closed, the data is lost.

This limitation is acceptable for the current project because the focus is not database implementation. In-memory repositories also make the system easier to test and demonstrate. However, in a real system, persistent storage would be necessary. Customer accounts, product stock, order history, payment records, and admin changes should remain available after the application restarts.

A future improvement would be to replace the in-memory repositories with database-backed repository implementations. Since the service layer depends on repository interfaces, this change could be made without changing most of the service logic.

9.4 Basic Authentication

The system includes a simple admin password mechanism. This is enough to separate customer operations from admin operations in the console application, but it is not a secure authentication system.

The current admin authentication does not include encrypted passwords, user roles, session management, password reset, or account locking. Also, customers log in using their e-mail address only. This is simple for demonstration, but it would not be acceptable for a real e-commerce platform.

A future improvement would be to implement a complete authentication and authorization system. Customers and admins could have usernames, encrypted passwords, roles, and secure login sessions. Admin permissions could also be divided into different roles such as product manager, order manager, and system administrator.

9.5 Simulated Payment Processing

The payment process is simulated. The system supports credit card, debit card, and wallet balance payments, but it does not connect to a real bank or payment provider. ExternalPaymentGateway is only a simulated external payment service.

This limitation is useful for the project because it allows the Adapter pattern to be demonstrated without depending on a real payment API. However, real payment systems require more complex features such as secure card storage rules, payment provider authentication, fraud checks, transaction logs, refunds, and legal compliance.

A future improvement would be to integrate a real sandbox payment provider. The current PaymentGateway abstraction and PaymentGatewayAdapter design would make this improvement easier because the internal payment logic is already separated from the external gateway details.

9.6 Basic Allergy Checking Logic

The allergy checking logic is based on direct matching between the customer's allergy profile and the product's allergen list. If a product contains an allergen that also exists

in the customer's allergy profile, the system displays a warning and asks for confirmation.

This is a useful and clear rule for the project, but real allergy safety is more complex. Real grocery systems may need to handle cross-contamination risks, severity levels, ingredient traces, "may contain" labels, alternative product suggestions, and medical restrictions. The current system does not evaluate these advanced cases.

A future improvement would be to add more advanced allergy checking strategies. For example, the system could support different risk levels such as low, medium, and high. It could also suggest safer alternative products. Since the project already uses the Strategy pattern for allergy checking, new allergy checking algorithms can be added more easily.

9.7 Limited Product and Inventory Features

The product management part of the system supports adding products, updating stock, disabling products, and viewing product information. However, it does not include advanced inventory management.

For example, the system does not support supplier management, product images, barcode scanning, expiration dates, discount campaigns, product reviews, or automatic low-stock alerts. Stock is decreased after successful checkout, but there is no warehouse or shipment management module.

A future improvement would be to add a separate inventory service. This service could manage stock reservations, stock history, low-stock warnings, and supplier-based replenishment. This would make the product management side closer to a real grocery system.

9.8 No Multi-User or Concurrency Support

The current application is designed for one console session at a time. It does not support multiple users using the system at the same time. Because of this, it does not handle concurrency problems.

In a real e-commerce system, multiple customers may try to buy the same product at the same time. This can create stock consistency problems if two checkout operations happen simultaneously. The current project does not include locking, transactions, or concurrency control.

A future improvement would be to add transaction management and concurrency-safe stock updates. If the system were connected to a database, stock updates could be handled with database transactions.

9.9 Limited Error Logging and Monitoring

The system displays error messages to the console, but it does not include a real logging or monitoring system. For example, failed payments, invalid inputs, and admin operations are not written to an external log file.

The project includes observer-based order notifications and logs inside the order object, but this is not the same as a complete system logging mechanism. A real application would need persistent logs for debugging, auditing, and security monitoring.

A future improvement would be to add a logging framework. Important operations such as login attempts, product updates, checkout failures, payment results, and order status changes could be stored in log files or a monitoring system.

9.10 Testing Limitations

The project can be tested manually through the console, and the code structure is suitable for unit testing. However, the current project does not include a complete automated test suite.

Manual testing is enough to demonstrate the main scenarios, but it is not as reliable as automated testing for long-term maintenance. If the system grows, manual testing may miss some edge cases or regression errors.

A future improvement would be to add JUnit tests for service classes, domain classes, payment processing, allergy checking, and order state transitions. Mock or fake repository implementations could be used to test services without depending on the console interface.

9.11 Limitations Summary

The main limitations of the system are related to interface, persistence, security, payment processing, allergy logic, inventory management, concurrency, logging, and automated testing. These limitations do not prevent the project from meeting its main educational goals. The system still demonstrates object-oriented analysis and design, layered architecture, design patterns, SOLID principles, and UML modeling.

The current design also leaves room for future development. Because the system is separated into layers and depends on abstractions in many places, features such as database storage, web interface, real payment integration, advanced allergy rules, and automated testing can be added more easily in future versions.

9.12 Additional Discussion: Complexity, Maintainability, and Architectural Choices

Although the main focus of this project is object-oriented analysis and design, it is still useful to discuss the system from the perspective of complexity, maintainability, extensibility, and architectural alternatives. These points help to show that the project was not designed only to work in the console, but also to demonstrate clear software design decisions.

9.12.1 Complexity Discussion

The Allergy Aware Grocery System is not a computation-heavy application. Most operations are related to validation, searching in in-memory collections, checking allergy information, updating cart items, processing simulated payment, and changing order status.

Product lookup by product id is efficient because the in-memory product repository stores products in a map-like structure. Therefore, finding a product by id is expected to be close to constant time in normal cases. Listing all products or active products requires visiting the product collection, so its time complexity is linear according to the number of products.

Listing all products: O(p)

Finding a product by id: approximately O(1)

Filtering active products: O(p)

Here, p represents the number of products in the system.

Customer login by e-mail is implemented by searching through the customer collection. Therefore, in the current in-memory implementation, finding a customer by e-mail has linear time complexity according to the number of customers.

Finding customer by e-mail: O(c)

Here, c represents the number of customers.

Cart operations are also mostly linear according to the number of cart items. For example, when a product is added to the cart, the cart checks whether the product already exists in the cart. This requires scanning the current cart items. Cart total calculation also visits each cart item once.

Adding an item to cart: O(i)

Removing an item by index: O(i) in an ArrayList because elements may shift

Calculating cart total: O(i)

Validating cart before checkout: O(i)

Decreasing stock after checkout: O(i)

Here, i represents the number of items in the cart.

Allergy checking compares the selected product's allergens with the customer's allergy profile. Since the allergen list is small and based on enum values, this operation is very fast in practice. The theoretical complexity depends on the number of allergens stored in the product.

Allergy check for one product: O(a)

Here, a represents the number of allergens assigned to the product.

Order status transition operations are simple because the State pattern delegates the transition to the current state object. The number of possible states is limited, so order transition logic is effectively constant time.

Order status transition: O(1)

Overall, the system has acceptable performance for the project scope. Since it is a console-based academic project with in-memory repositories, performance is not a major bottleneck. If the system were converted into a real e-commerce application, database queries, indexing, concurrency, caching, and transaction management would become more important.

9.12.2 Maintainability and Extensibility Discussion

The system is designed to be maintainable by separating responsibilities across layers and classes. The presentation layer handles console interaction. The application/service layer coordinates use cases. The domain layer contains business models, rules, interfaces, states, strategies, observers, factories, and value objects.

The infrastructure layer contains technical details such as in-memory repositories, payment adapter implementation, application configuration, and initial product seeding.

This structure makes the code easier to modify because changes are usually localized. For example, if customer registration validation changes, the main class to update is CustomerService. If payment validation changes, the main class to update is PaymentService. If a new external payment provider is added, a new adapter can be implemented without changing the whole checkout process.

Extensibility is also supported by the design patterns used in the project. The Strategy pattern allows new allergy checking algorithms to be added. The Observer pattern allows new reactions to order status changes. The State pattern allows order lifecycle rules to be managed in separate state classes. The Adapter pattern allows external payment systems to be integrated. The Facade pattern keeps checkout coordination clear for the presentation layer.

Possible future extensions include:
- Adding a web or GUI-based presentation layer
- Replacing in-memory repositories with database repositories
- Adding advanced allergy checking strategies
- Adding product recommendations
- Adding e-mail or SMS order notifications
- Adding real payment gateway integration
- Adding JUnit tests for service and domain classes
- Adding role-based admin authorization

The current design supports these future improvements better than a single-class or procedural implementation would. Because service classes depend on interfaces in important areas, many infrastructure changes can be made with limited effect on the application logic.

9.12.3 Manual Dependency Injection

The project uses manual dependency injection instead of a dependency injection framework. ApplicationContext creates repository implementations, services, factories, gateways, and facades, and then passes dependencies through constructors.

This approach is suitable for the project because the system is not large enough to require a full dependency injection framework. Manual dependency injection also

makes the object creation process visible and understandable. It clearly shows which services depend on which repositories or other services.

For example, PaymentService receives a PaymentGateway dependency instead of creating a concrete payment gateway by itself. CartService receives cart and product repositories and the allergy check service. CheckoutFacade receives the services needed to coordinate checkout.

This design improves testability because dependencies can be replaced with fake or test implementations when unit tests are written. It also supports the Dependency Inversion Principle because high-level service classes depend on abstractions instead of directly depending on low-level details.

The main limitation is that ApplicationContext can grow as the number of dependencies increases. In a larger project, a framework such as Spring could manage dependency creation and injection. However, for this academic console project, manual dependency injection is clear, simple, and appropriate.

9.12.4 Alternative Architectural Choices

Several alternative architectural choices were possible for this project. The first alternative was to implement the system as a simple procedural console program. This would have been faster to write, but it would not demonstrate object-oriented design quality. It would also make the code harder to extend because menu logic, validation, business rules, and storage operations would likely be mixed together.

The second alternative was to use a full web application architecture with controllers, database access, authentication, and a user interface. This would be more realistic for an e-commerce system, but it would also increase implementation complexity. Since the term project focuses on OOAD, layered architecture, design patterns, SOLID principles, and UML modeling, a console-based layered system is a better fit for the project scope.

The third alternative was to connect the system to a real database. A database would provide permanent storage and more realistic data management. However, the project requirements allow in-memory repositories, and the current design already separates repository interfaces from repository implementations. Therefore, using in-memory repositories keeps the project simple while still preserving architectural separation.

Another alternative was to manage order statuses with a simple enum and if-else statements. This would reduce the number of classes, but it would make transition

logic more centralized and less extensible. The State pattern was selected instead because order lifecycle management is an important business rule in the system.

Similarly, allergy checking could have been implemented directly inside CartService. However, this would make CartService responsible for both cart management and allergy decision logic. The Strategy pattern was selected instead because allergy checking may change in future versions.

Overall, the chosen architecture balances project complexity and design quality. It is simpler than a full enterprise application, but it still demonstrates layered architecture, object-oriented responsibilities, design patterns, SOLID principles, and maintainable code structure.

9.13 Reflection and Future Improvements

During the project, one of the main challenges was balancing simplicity and design quality. Since the system is a console-based academic project, it had to remain understandable and runnable without a database, GUI framework, or external payment provider. At the same time, it also had to demonstrate object-oriented design, layered architecture, design patterns, SOLID principles, and UML modeling.

Another challenge was deciding where each responsibility should belong. For example, putting all checkout logic inside the menu class would have been easier at first, but it would have created a weak design. For this reason, checkout coordination was moved into CheckoutFacade, payment logic was placed in PaymentService, allergy checking was separated with Strategy, and order lifecycle behavior was controlled with the State pattern.

The project also showed the importance of keeping diagrams and code consistent. Use cases, sequence diagrams, class diagrams, and package diagrams were prepared according to the implemented Java structure. This helped make the report and implementation support each other.

Future improvements can include adding a web-based user interface, replacing in-memory repositories with database-backed repositories, implementing full customer password authentication, adding automated JUnit tests, supporting more advanced allergy checking rules, adding product search and filtering, integrating a real payment sandbox, and improving admin authorization. These improvements can be added more easily because the current system already separates presentation, application, domain, and infrastructure responsibilities.

10. Conclusion

The Allergy Aware Grocery System was developed as a medium-scale Java console application to demonstrate object-oriented analysis and design principles. The main purpose of the system is to support grocery shopping for customers who have food allergies. Customers can register with an allergy profile, browse active products, receive allergy warnings, manage their cart, and complete checkout with simulated payment options. Admin users can manage products, stock values, customers, orders, and order status transitions.

The project follows a layered architecture. The presentation layer is responsible for console interaction. The application/service layer coordinates use cases such as registration, cart management, checkout, payment, and admin operations. The domain layer contains the main business models and rules. The infrastructure layer contains in-memory repository implementations, payment adapter classes, product seeding, and application configuration. This separation makes the system easier to understand, maintain, and extend.

The project also applies several design patterns. Singleton is used in ApplicationContext, Builder is used in CustomerBuilder, and Factory Method is used in ProductFactory. Facade is used in CheckoutFacade to simplify the checkout process. Adapter is used in PaymentGatewayAdapter to connect the internal payment model with the simulated external payment gateway. Strategy is used for allergy checking, Observer is used for order notifications and logs, and State is used for order lifecycle management. These patterns are not used only for demonstration; each one solves a specific design problem in the system.

The UML diagrams support the design explanation. The use case diagram shows the functional scope of the system. The package diagram shows the layered structure. The class diagram shows the main classes, interfaces, enums, and relationships. The sequence diagrams explain the runtime interactions for registration, allergy-aware add-to-cart, checkout, and admin order status update. The activity diagrams show decision-based business workflows, and the statechart diagram shows the order lifecycle.

The SOLID evaluation shows that the system generally follows object-oriented design principles. Responsibilities are separated between services, domain objects, repositories, strategies, states, observers, factories, and adapters. The system depends on abstractions in important areas, especially repositories, payment gateway, allergy strategy, order state, and observers. Some improvements are still possible, especially in the console menu classes and some conditional service logic, but the overall structure is clean and understandable for the project scope.

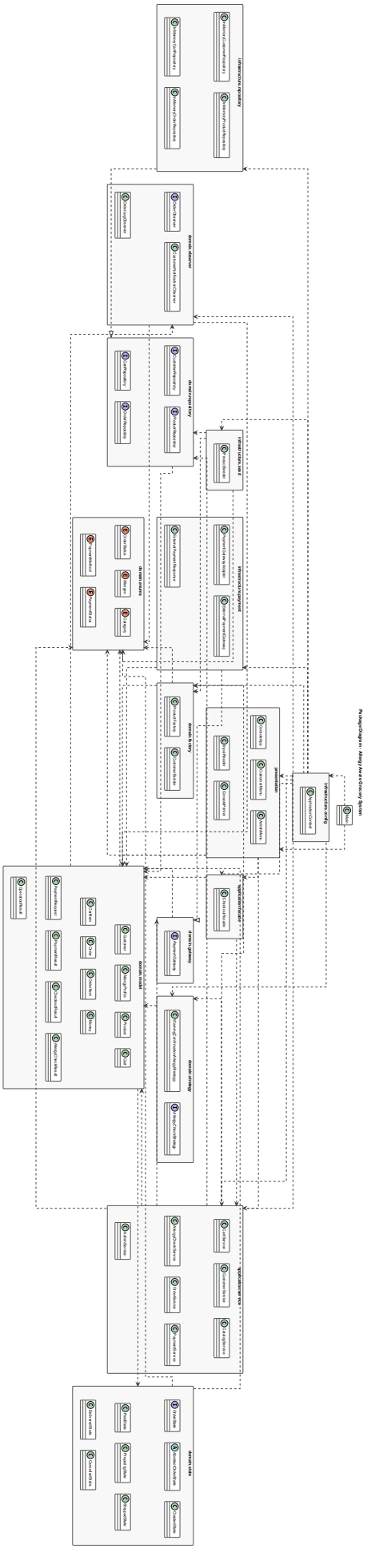
Testing was planned with scenario-based manual testing and possible unit testing. The main tested flows include customer registration, login, product browsing, allergy

checking, cart operations, checkout, payment processing, order status transitions, and admin operations. The project is suitable for future JUnit tests because many responsibilities are separated into service and domain classes.

The system has some limitations. It uses a console-based interface, in-memory storage, simple authentication, simulated payment processing, and basic allergy matching logic. It does not include a real database, web interface, real payment provider, advanced medical allergy validation, or concurrency support. However, these limitations are acceptable because the main goal of the project is to demonstrate software design quality rather than build a production-ready grocery platform.

In conclusion, the Allergy Aware Grocery System meets the main goals of the term project. It demonstrates layered architecture, object-oriented design, UML modeling, design patterns, SOLID principles, clean code practices, and maintainability considerations. The current implementation is simple enough to understand as an academic project, but structured enough to be extended with more realistic features in the future.

Appendix A



Appendix B

